

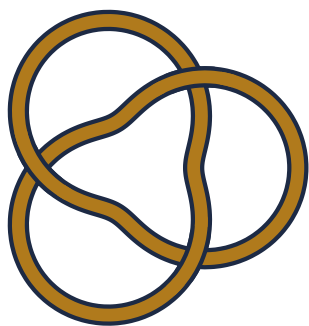


nibart

Calligraphie, nœuds et ornements — une plume « à la MetaPost » pour Typst

Manuel de référence Version 0.3.0 · Typst ≥ 0.15 · MIT

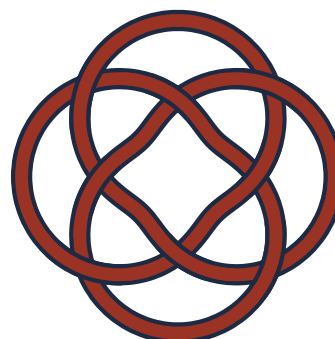
Auteur FERGOUS Abdelhak <@fergousA> · github.com/fergousA/nibart



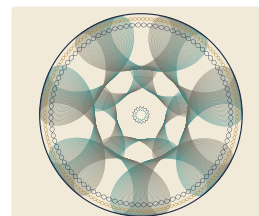
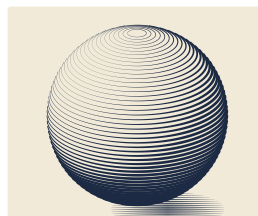
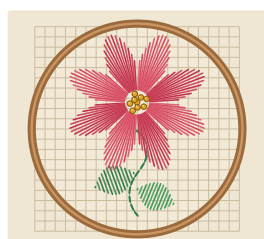
3 croisements



5 croisements



8 croisements



Sommaire

Prise en main	3
Ce que fait nibart	3
Installation	3
Démarrage rapide	3
Concepts	4
Comment lire ce manuel	4
Chemins et géométrie	5
Construire des chemins	5
Mesurer et interroger	12
Découper et modifier	18
Transformations	23
Plumes	27
Plumes fixes	27
La plume calligraphique et ses options	29
Dessiner	32
Traits	32
Remplissages, points, étiquettes	34
Figures	36
Nœuds, ornements et décorations	38
Nœuds et entrelacs	38
Décorations calligraphiques	40
Version	41
Recettes	43
Boucle façon écriture à la plume plate	43
Une flèche courbe	43
Un collier de perles sur une courbe fermée	43
Croisements dessus/dessous à la main	43
Reprendre un dessin écrit pour nibst	44
Notes et limites	45
Crédits et licence	46
Index des fonctions	47

Prise en main

Ce que fait nibart

`nibart` trace des traits comme Metafont et MetaPost : vous décrivez un **chemin** (algorithme de Hobby, directions, tensions, « curl »...) et une **plume** (cercle, ellipse, plume plate, polygone, plume dont la forme change le long du chemin), et `nibart` calcule le contour exact balayé par la plume. Le résultat est une forme vectorielle Typst ordinaire. Au-dessus de ce moteur, le paquet propose des **nœuds et entrelacs**, des **décorations calligraphiques** (accolades, traits à la plume pointue, plumes à plusieurs dents) et de la **chirurgie de chemins** (découper, décaler, disposer des objets le long d'une courbe).

La géométrie est calculée par un petit plugin Rust compilé en WebAssembly (`nibart.wasm`), livré dans le paquet. Aucune police, aucun programme externe, aucun réseau.

Installation

```
#import "@preview/nibart:0.3.0" as nib
```

Typst Universe : rien à installer. Une édition locale (zip avec `install.sh` / `install.ps1`) installe le même paquet dans l'espace de noms `@local`.

Préférez la forme qualifiée `nib.xxx` : le paquet exporte des noms génériques (`draw`, `cubic`, `reverse`, `pressure`, `shifted`...) qui masqueraient vos propres définitions avec `import "...": *`. Vous pouvez aussi n'importer que le nécessaire : `#import "@preview/nibart:0.3.0": mp-path, nib-stroke, broadnib`.

Démarrage rapide

Un chemin, une plume, un appel :

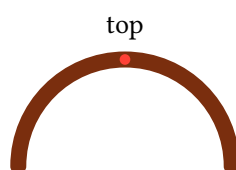
```
#import "@preview/nibart:0.3.0" as nib

#nib.nib-stroke(
  nib.mp-path("(0,0)..(40,30)..(80,0)..(120,30)"),
  pen: nib.broadnib(10pt, t: 2.5pt, angle: 30deg),
)
```



Pour plus de contrôle, construisez vous-même les objets dessinables et placez-les dans une figure :

```
#nib.mp-fig(
  nib.draw(nib.mp-path("(0,0){up}..(40,40)..(80,0)"),
    pen: nib.pencircle(6pt), fill: rgb("#7a2e0e")),
  nib.mp-dot((40pt, 40pt), pen: nib.pencircle(4pt), fill: red),
  nib.mp-label([top], (40pt, 50pt), align: bottom),
  pad: 4pt,
)
```



Concepts

- **Axes.** Comme en MetaPost, **y est orienté vers le haut** et les angles sont anti-horaires. Dans un `mp-fig`, l'origine est celle que vous choisissez (par défaut le contenu est ajusté à sa boîte englobante).
- **Unités.** Tout argument peut être une longueur ou un angle Typst (`3mm` ; `1em` est refusé car il dépend du contexte) ou un nombre simple (lu en pt, ou en degrés pour un angle). Les chaînes de chemin (`mp-path`) lisent les nombres comme des multiples de `unit` (défaut `1pt`).
- **Chemins.** Valeurs opaques (liste de segments de Bézier), créées par `mp-path` (syntaxe MetaPost), `mp-path-pts`, `cubic`, `straight`, `polyline`, `cubics` ; assemblées par `path-join` ; transformées par `shifted`, `scaled`, `rotated`...
- **Temps.** Un chemin de `n` segments a des temps de `0` à `n`. Le temps `2.5` est le milieu du troisième segment — comme `point 2.5 of p` en MetaPost.
- **Plumes.** De simples dictionnaires : fixes (`penellipse`, `pencircle`, `broadnib`, `penpoly`, `pensquare`, `penrazor`) ou variables (`nibpen`).
- **Objets dessinables** — créés par `draw`, `stroke-items`, `mp-fill`, `mp-dot`, `mp-label`, `mp-skeleton`, `mp-group` et par les fonctions d'ornement de la seconde partie — qu'on confie à `mp-fig`, lequel renvoie une `box` ordinaire. `nib-stroke` est le raccourci pour un seul trait.
- **Formes, pas traits.** `draw` et ses semblables renvoient des formes **remplies** (la région balayée par la plume) : `fill` est donc la couleur de l'encre.

Comment lire ce manuel

La référence ci-dessous décrit chaque fonction publique de la même façon : sa **signature** avec tous les paramètres et leurs valeurs par défaut ; un **tableau des paramètres** (nom, type, défaut, signification — « obligatoire » signifie sans valeur par défaut) ; ce qu'elle **renvoie** ; et un **exemple** détaillé, avec le code à gauche et le résultat réel à droite. Les commentaires des exemples expliquent chaque étape.

Les exemples sont écrits en **mode code** et utilisent les noms **sans** le préfixe `nib.`, comme après `#import "@preview/nibart:0.3.0": *`. Dans votre document, importez les noms nécessaires ou préfixez-les par `nib.`, et placez le code dans `#{ ... }` ou dans le corps d'une fonction. La valeur de la dernière expression est ce qui s'affiche à droite. Un **index de toutes les fonctions**, avec numéros de page, clôt le manuel.

Chemins et géométrie

Construire des chemins

Un **chemin** est une valeur opaque : une liste de segments de Bézier cubiques, ouverte ou cyclique. Chaque fonction ci-dessous en renvoie un. Les coordonnées sont des longueurs Typst (un nombre seul vaut des pt) ; **y est orienté vers le haut**.

mp-path

```
mp-path(spec, unit: 1pt)
```

Analyse un chemin écrit dans le langage MetaPost et calcule la courbe lisse par l'algorithme de Hobby. Syntaxe prise en charge : points `(x,y)` ; liaisons `..` (courbe de Hobby), `...` (traité comme `..`) et `--` (droite) ; directions `{up}`, `{down}`, `{left}`, `{right}`, `{dir 45}`, `{x,y}`, `{curl 2}` ; tension `a` et tension `a and b` ; controls `P` et controls `P and Q` ; cycle pour fermer le chemin.

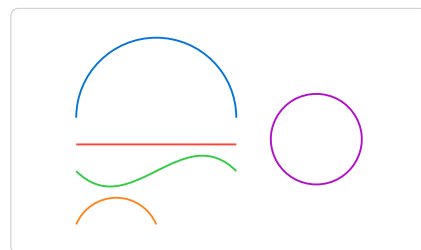
PARAMÈTRES

nom	type	défaut	description
spec	chaîne	obligatoire	La spécification du chemin.
unit	longueur	1pt	Unité par laquelle sont multipliés tous les nombres de spec.

RENVOIE un chemin.

EXEMPLE

```
// quatre spécifications, tracées en squelette
let a = mp-path("(0,0)..(30,30)..
(60,0)") // courbe de Hobby simple
// direction imposée + tension
let b = mp-path("(0,-10){right}..tension 2..(60,-10)")
// contrôles de Bézier explicites
let c = mp-path("(0,-20)..controls (20,-40) and (40,0)..
(60,-20)")
let d = mp-path("(75,0)..(105,0)..
(90,-25)..cycle") // chemin fermé
// nombres multipliés par `unit`
let e = mp-path("(0,0)..(3,2)..(6,0)", unit: 5pt)
mp-fig(
  mp-skeleton(a, stroke: 0.7pt + blue),
  mp-skeleton(b, stroke: 0.7pt + red),
  mp-skeleton(c, stroke: 0.7pt + green),
  mp-skeleton(d, stroke: 0.7pt + purple),
  mp-skeleton(shifted(e, 0pt, -40pt), stroke: 0.7pt +
orange),
  pad: 4pt)
```



mp-path-pts

```
mp-path-pts(pts, cycle: false, tension: 1, curl: none)
```

Chemin lisse (Hobby) passant par une liste de points. Pratique quand les points sont calculés dans une boucle et qu'on ne veut pas fabriquer de chaîne de spécification.

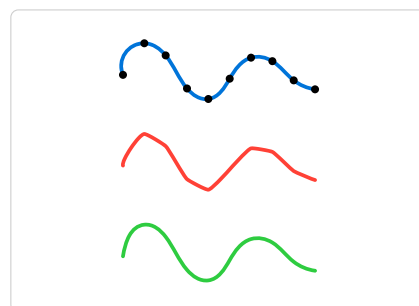
PARAMÈTRES

nom	type	défaut	description
<code>pts</code>	tableau de points	obligatoire	Points (x, y) , chaque coordonnée étant une longueur.
<code>cycle</code>	booléen	<code>false</code>	Ferme le chemin de façon lisse.
<code>tension</code>	nombre	<code>1</code>	Tension de chaque liaison (comme <code>..tension t..</code>); plus grand = courbe plus tendue.
<code>curl</code>	nombre ou none	<code>none</code>	« Curl » au premier point d'un chemin ouvert (<code>{curl c}</code>); <code>0</code> = extrémité plate.

RENVOIE un chemin.

EXEMPLE

```
// dix points d'une onde amortie
let pts = range(10).map(i =>
  (i * 8pt, 14pt * calc.sin(i * 70deg) * calc.pow(0.9,
i)))
// same points, three settings, stacked
// les mêmes points, trois réglages, empilés
let loose = mp-path-pts(pts)
let tight = shifted(mp-path-pts(pts, tension: 3), 0pt,
-34pt)
let flat = shifted(mp-path-pts(pts, curl: 0), 0pt, -68pt)
mp-fig(
  draw(loose, pen: pencircle(1.4pt), fill: blue),
  draw(tight, pen: pencircle(1.4pt), fill: red),
  draw(flat, pen: pencircle(1.4pt), fill: green),
  ..pts.map(q => mp-dot(q, pen: pencircle(3pt))),
  pad: 4pt)
```



cubic

`cubic(a, b, c, d)`

Un segment de Bézier cubique de `a` à `d`, avec les points de contrôle `b` et `c`.

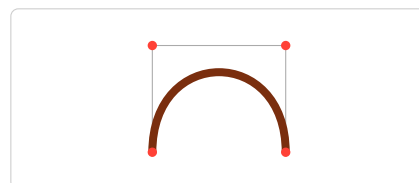
PARAMÈTRES

nom	type	défaut	description
<code>a</code>	point	obligatoire	Point de départ (x, y) .
<code>b</code>	point	obligatoire	Premier point de contrôle.
<code>c</code>	point	obligatoire	Second point de contrôle.
<code>d</code>	point	obligatoire	Point d'arrivée.

RENVOIE un chemin ouvert d'un segment.

EXEMPLE

```
let (a, b, c, d) = ((0pt, 0pt), (0pt, 40pt), (50pt,
40pt), (50pt, 0pt))
let p = cubic(a, b, c, d)
mp-fig(
  // polygone de contrôle
  mp-skeleton(polyline((a, b, c, d)), stroke: 0.4pt +
gray),
  draw(p, pen: pencircle(3pt), fill: rgb("#7a2e0e")),
  ..(a, b, c, d).map(q => mp-dot(q, pen:
pencircle(3.5pt), fill: red)),
  pad: 5pt)
```



straight

```
straight(a, b)
```

Un segment droit, stocké comme une cubique dégénérée pour se combiner librement avec des courbes.

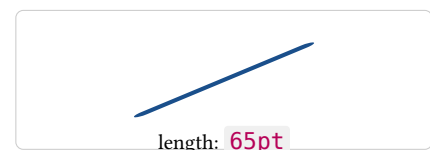
PARAMÈTRES

nom	type	défaut	description
a	point	obligatoire	Point de départ.
b	point	obligatoire	Point d'arrivée.

RENVOIE un chemin ouvert d'un segment.

EXEMPLE

```
let s = straight((0pt, 0pt), (60pt, 25pt))
mp-fig(
  draw(s, pen: broadnib(8pt, t: 1.5pt, angle: 20deg),
  fill: rgb("#1a4f8b")),
  // longueur mesurée
  mp-label(text(8pt)[length: #arclength(s)], (30pt,
  -10pt)),
  pad: 5pt)
```



polyline

```
polyline(pts, cycle: false)
```

Chemin polygonal passant par les points (segments droits).

PARAMÈTRES

nom	type	défaut	description
pts	tableau de points	obligatoire	Les sommets.
cycle	booléen	false	Ferme le polygone.

RENVOIE un chemin (un segment par côté).

EXEMPLE

```
// pentagramme régulier
let pt(i) = (26pt * calc.cos(90deg + i * 144deg),
  26pt * calc.sin(90deg + i * 144deg))
let star = polyline(range(5).map(pt), cycle: true)
mp-fig(
  draw(star, pen: penrazor(3pt, angle: 45deg), fill:
  rgb("#9a3324")),
  pad: 4pt)
```



cubics

```
cubics(segments, cycle: false, y-down: false)
```

Chemin à partir d'une liste de segments cubiques explicites ((p0, c1, c2, p3), ...). La fin de chaque segment doit être le début du suivant. Avec `y-down: true`, les coordonnées suivent la convention Typst (y vers le bas) : le chemin est retourné et les angles de plume sont inversés, ce qui reproduit les dessins écrits pour le paquet `nibst`.

PARAMÈTRES

nom	type	défaut	description
<code>segments</code>	tableau	obligatoire	Tableau de quadruplets <code>(p0, c1, c2, p3)</code> .
<code>cycle</code>	booléen	<code>false</code>	Marque le chemin comme fermé.
<code>y-down</code>	booléen	<code>false</code>	Interprète les coordonnées avec y vers le bas.

RENVOIE un chemin.

EXEMPLE

```
// un S fait de deux cubiques
let s = cubics((
  ((0pt, 0pt), (30pt, 0pt), (40pt, 20pt), (20pt,
    24pt)),
  ((20pt, 24pt), (0pt, 28pt), (10pt, 48pt), (40pt,
    48pt)),
))
nib-stroke(s, pad: 4pt, fill: rgb("#2b2b2b"),
  pen: nibpen(width: 7pt, thinness: 25%, angle: 40deg))
```



path-join

```
path-join(a, b)
```

Concatène deux chemins ouverts (la fin de `a` est supposée être le début de `b` ; aucune vérification).

PARAMÈTRES

nom	type	défaut	description
<code>a</code>	chemin	obligatoire	Premier chemin.
<code>b</code>	chemin	obligatoire	Second chemin.

RENVOIE un chemin ouvert.

EXEMPLE

```
let a = mp-path("(0,0)..(20,20)")
let b = mp-path("(20,20)..(40,0)..(60,20)")
mp-fig(
  draw(path-join(a, b), pen: pencircle(3pt), fill:
    rgb("#1a4f8b")),
  pad: 4pt)
```



path-join-all

```
path-join-all(parts, cycle: false)
```

Concatène toute une liste de chemins, avec fermeture optionnelle du résultat.

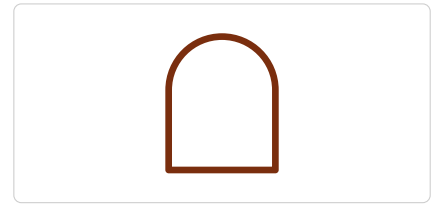
PARAMÈTRES

nom	type	défaut	description
<code>parts</code>	tableau de chemins	obligatoire	Chemins à enchaîner, dans l'ordre.
<code>cycle</code>	booléen	<code>false</code>	Ferme le résultat.

RENVOIE un chemin.

EXEMPLE


```
// une « stèle » : deux droites + un arc
let up = straight((0pt, 0pt), (0pt, 30pt))
let arc = mp-path("(0,30){up}..(20,50){right}..{down}(40,30)")
let down = straight((40pt, 30pt), (40pt, 0pt))
let shape = path-join-all((up, arc, down), cycle: true)
mp-fig(draw(shape, pen: pencircle(2.5pt), fill:
rgb("#7a2e0e")), pad: 4pt)
```



path-close

```
path-close(p)
```

Marque un chemin ouvert comme cyclique : on sous-entend un segment droit de la fin au début.

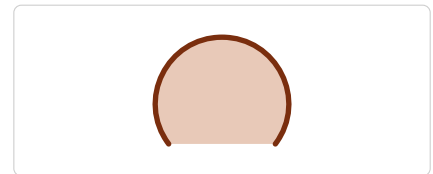
PARAMÈTRES

nom	type	défaut	description
p	chemin	obligatoire	Chemin ouvert.

RENVOIE un chemin fermé.

EXEMPLE

```
let open = mp-path("(0,0)..(20,40)..(40,0)")
mp-fig(
  // remplissage grâce à la fermeture
  mp-fill(path-close(open), fill: rgb("#e8c9b8")),
  draw(open, pen: pencircle(2pt), fill: rgb("#7a2e0e")),
  pad: 4pt)
```



round-corners

```
round-corners(pts, r: 4pt, cycle: false)
```

Polygone dont les coins sont remplacés par des arcs de cercle de rayon **r** (réduit automatiquement quand les côtés sont trop courts).

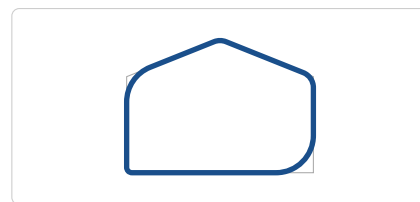
PARAMÈTRES

nom	type	défaut	description
pts	tableau de points	obligatoire	Sommets du polygone.
r	longueur ou tableau	4pt	Rayon, ou un rayon par sommet.
cycle	booléen	false	Polygone fermé (tous les coins arrondis) ou ouvert (extrémités conservées).

RENVOIE un chemin.

EXEMPLE

```
let pts = ((0pt, 0pt), (70pt, 0pt), (70pt, 36pt), (35pt, 50pt), (0pt, 36pt))
mp-fig(
  mp-skeleton(polyline(pts, cycle: true), stroke: 0.4pt + gray),
  draw(
    round-corners(pts, r: (2pt, 14pt, 6pt, 6pt, 14pt),
    cycle: true),
    pen: pencircle(2pt), fill: rgb("#1a4f8b")),
    pad: 4pt)
```



join-smooth

```
join-smooth(a, b, tension: 1)
```

Raccorde la fin de `a` au début de `b` par une courbe de Hobby qui quitte `a` et entre dans `b` selon leurs tangentes (pas d'angle à la jonction).

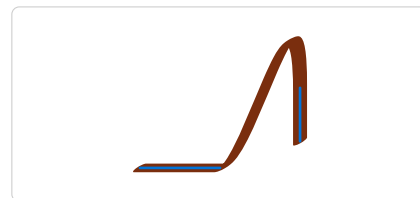
PARAMÈTRES

nom	type	défaut	description
<code>a</code>	chemin	obligatoire	Chemin ouvert à quitter.
<code>b</code>	chemin	obligatoire	Chemin ouvert à rejoindre.
<code>tension</code>	nombre	1	Tension de la courbe de raccord.

RENVOIE un chemin ouvert `a` + raccord + `b`.

EXEMPLE

```
let a = mp-path("(0,0){right}..(30,0)")
let b = mp-path("(60,30){down}..(60,10)")
mp-fig(
  draw(join-smooth(a, b), pen: broadnib(6pt, t: 1.4pt,
    angle: 30deg), fill: rgb("#7a2e0e")),
  draw(a, pen: pencircle(1pt), fill: blue), draw(b, pen:
    pencircle(1pt), fill: blue),
    pad: 4pt)
```



close-smooth

```
close-smooth(p, tension: 1)
```

Ferme un chemin ouvert par une courbe lisse de sa fin à son début, en respectant les deux tangentes.

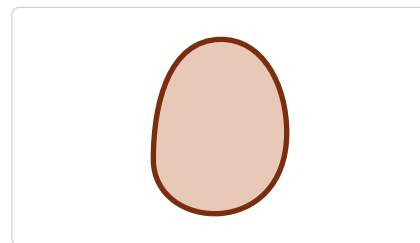
PARAMÈTRES

nom	type	défaut	description
<code>p</code>	chemin	obligatoire	Chemin ouvert.
<code>tension</code>	nombre	1	Tension de la courbe de fermeture.

RENVOIE un chemin fermé.

EXEMPLE

```
let p = mp-path("(0,0){up}..(25,45)..{down}(50,10)")
mp-fig(
  mp-fill(close-smooth(p), fill: rgb("#e8c9b8")),
  draw(close-smooth(p), pen: pencircle(2pt), fill:
    rgb("#7a2e0e")),
  pad: 4pt)
```



brace-path

```
brace-path(length, amplitude)
```

Chemin nu d'une accolade calligraphique de $(0,0)$ à $(length,0)$, bombée vers $y > 0$, sa pointe à la hauteur `amplitude`. Voir `delimenter` pour la tracer avec une plume qui gonfle.

PARAMÈTRES

nom	type	défaut	description
<code>length</code>	longueur	obligatoire	Distance entre les deux extrémités.
<code>amplitude</code>	longueur	obligatoire	Hauteur de la pointe.

RENVOIE un chemin ouvert.

EXEMPLE

```
let b = brace-path(80pt, 10pt)
mp-fig(
  draw(b, pen: pencircle(1.6pt), fill: rgb("#9a3324")),
  mp-skeleton(straight((0pt, 0pt), (80pt, 0pt)), stroke:
    0.4pt + gray),
  pad: 4pt)
```



paren-path

```
paren-path(length, amplitude, straight: false)
```

Chemin nu d'un arc en forme de parenthèse de $(0,0)$ à $(length,0)$, bombé vers $y > 0$. Avec `straight: true`, les deux extrémités courbes sont reliées par un milieu droit (longue parenthèse).

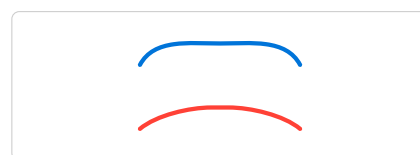
PARAMÈTRES

nom	type	défaut	description
<code>length</code>	longueur	obligatoire	Distance entre les extrémités.
<code>amplitude</code>	longueur	obligatoire	Hauteur du bombé.
<code>straight</code>	booléen	<code>false</code>	Partie centrale droite.

RENVOIE un chemin ouvert.

EXEMPLE

```
mp-fig(
  draw(paren-path(60pt, 8pt), pen: pencircle(1.6pt),
    fill: blue),
  draw(shifted(paren-path(60pt, 8pt, straight: true),
    0pt, -24pt),
    pen: pencircle(1.6pt), fill: red),
  pad: 4pt)
```



Mesurer et interroger

Les chemins sont paramétrés par le **temps** : un chemin de `n` segments a des temps de `0` à `n` ; le temps `2.5` est le milieu du troisième segment (comme `point 2.5 of p` en MetaPost). Les positions renvoyées sont des longueurs Typst, les angles des angles Typst.

path-length

`path-length(p)`

Nombre de segments de Bézier du chemin, c'est-à-dire son temps maximal. (Pour la longueur géométrique, voir `arclength`.)

PARAMÈTRES

nom	type	défaut	description
<code>p</code>	<code>chemin</code>	obligatoire	Un chemin quelconque.

RENVOIE un entier.

EXEMPLE

```
let p = mp-path("(0,0)..(30,30)..(60,0)..(90,30)")
let q = mp-path("(0,0)..(30,30)..(60,0)..cycle")
[open: #path-length(p) segments \
closed: #path-length(q) segments]
```

open: 3 segments
closed: 3 segments

arclength

`arclength(p)`

Longueur d'arc totale du chemin.

PARAMÈTRES

nom	type	défaut	description
<code>p</code>	<code>chemin</code>	obligatoire	Un chemin quelconque.

RENVOIE une longueur.

EXEMPLE

```
let half = mp-path("(0,0){right}..(40,40){left}")
let full = mp-path("(0,0)..(40,40)..(0,80)..
(-40,40)..cycle")
[arc: #calc.round(arclength(half).pt(), digits: 2) pt \
loop: #calc.round(arclength(full).pt(), digits: 2) pt]
```

arc: 81.29 pt
loop: 251.36 pt

arctime

`arctime(p, len)`

Temps auquel le chemin a parcouru la longueur d'arc `len` (`arctime len of p` de MetaPost). Le résultat peut être donné à `point-of`, `subpath`...

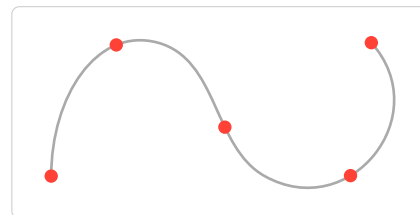
PARAMÈTRES

nom	type	défaut	description
<code>p</code>	<code>chemin</code>	obligatoire	Chemin.
<code>len</code>	<code>longueur</code>	obligatoire	Distance parcourue le long du chemin.

RENVOIE un nombre (temps).

EXEMPLE

```
let p = mp-path("(0,0){up}..(40,50)..(80,0)..(120,50)")
let L = arclength(p)
// points every quarter of the length
// points à chaque quart de la longueur
let marks = (0, 0.25, 0.5, 0.75, 1).map(f =>
  point-of(p, arctime(p, L * f)))
mp-fig(
  draw(p, pen: pencircle(1pt), fill: gray),
  ..marks.map(m => mp-dot(m, pen: pencircle(5pt), fill:
    red)),
  pad: 4pt)
```



point-of

```
point-of(p, t)
```

Point du chemin au temps `t` (`point t of p`). Sur un chemin cyclique, les temps se prolongent en boucle.

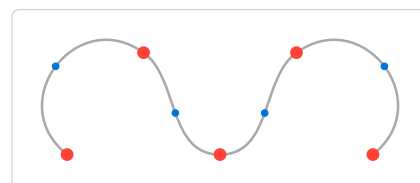
PARAMÈTRES

nom	type	défaut	description
<code>p</code>	<code>chemin</code>	obligatoire	Chemin.
<code>t</code>	<code>nombre</code>	obligatoire	Temps : indice de segment + fraction.

RENVOIE un point `(x, y)` en longueurs.

EXEMPLE

```
let p = mp-path("(0,0)..(30,40)..(60,0)..(90,40)..
(120,0)")
mp-fig(
  draw(p, pen: pencircle(1pt), fill: gray),
  // integer times = the nodes; halves = mid-segments
  // temps entiers = les nœuds ; demis = milieux
  ..range(9).map(i => mp-dot(point-of(p, i / 2),
    pen: pencircle(if calc.even(i) { 5pt } else
    { 3pt })),
    fill: if calc.even(i) { red } else { blue })),
  pad: 4pt)
```



direction-of

```
direction-of(p, t)
```

Vecteur tangent du chemin au temps `t` (non normalisé ; composantes en pt).

PARAMÈTRES

nom	type	défaut	description
<code>p</code>	<code>chemin</code>	obligatoire	Chemin.
<code>t</code>	<code>nombre</code>	obligatoire	Temps.

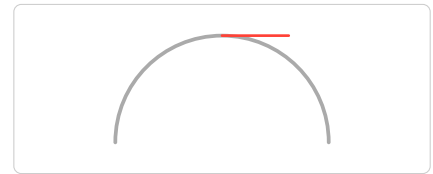
RENVOIE un tableau `(dx, dy)` de nombres.

EXEMPLE

```

let p = mp-path("(0,0)..(40,40)..(80,0)")
let d = direction-of(p, 1)
let at = point-of(p, 1)
// draw the tangent at the top, scaled to a fixed length
// trace la tangente au sommet, à longueur fixe
let n = calc.sqrt(d.at(0) * d.at(0) + d.at(1) * d.at(1))
let e = (at.at(0) + d.at(0) / n * 25pt, at.at(1) +
d.at(1) / n * 25pt)
mp-fig(
  draw(p, pen: pencircle(1.4pt), fill: gray),
  draw(straight(at, e), pen: pencircle(1pt), fill: red),
  pad: 4pt)

```



direction-angle

`direction-angle(p, t)`

Angle de la tangente au temps `t`, anti-horaire depuis l'axe x.

PARAMÈTRES

nom	type	défaut	description
<code>p</code>	chemin	obligatoire	Chemin.
<code>t</code>	nombre	obligatoire	Temps.

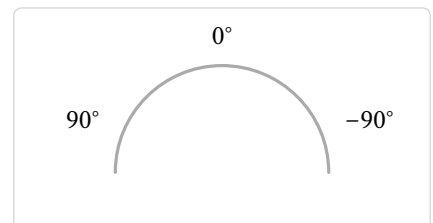
RENVOIE un angle.

EXEMPLE

```

let p = mp-path("(0,0){up}..(40,40){right}..(80,0)
{down}")
let ang(t) = [#calc.round(direction-angle(p, t).deg())°]
mp-fig(
  draw(p, pen: pencircle(1.2pt), fill: gray),
  mp-label(ang(0), (-6pt, 20pt), align: right),
  mp-label(ang(1), (40pt, 48pt), align: bottom),
  mp-label(ang(2), (86pt, 20pt), align: left),
  pad: 14pt)

```



path-bbox

`path-bbox(p)`

Boîte englobante du chemin (de la courbe elle-même, pas d'une plume).

PARAMÈTRES

nom	type	défaut	description
<code>p</code>	chemin	obligatoire	Chemin.

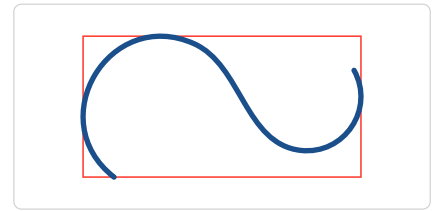
RENVOIE un dictionnaire `(x0:, y0:, x1:, y1:)` de longueurs.

EXEMPLE

```

let p = mp-path("(0,0)..(30,50)..(70,10)..(90,40)")
let b = path-bbox(p)
// rectangle drawn from the box
// rectangle tracé d'après la boîte
let r = polyline(((b.x0, b.y0), (b.x1, b.y0),
(b.x1, b.y1), (b.x0, b.y1)), cycle: true)
mp-fig(
  mp-skeleton(r, stroke: 0.6pt + red),
  draw(p, pen: pencircle(2pt), fill: rgb("#1a4f8b")),
  pad: 4pt)

```



closest-time

```
closest-time(p, pos)
```

Temps du point du chemin le plus proche de `pos`, et distance à ce point.

PARAMÈTRES

nom	type	défaut	description
<code>p</code>	<code>chemin</code>	obligatoire	Chemin.
<code>pos</code>	<code>point</code>	obligatoire	Point de référence <code>(x, y)</code> .

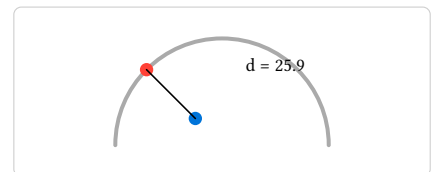
RENVOIE un tableau `(t, distance)`.

EXEMPLE

```

let p = mp-path("(0,0)..(40,40)..(80,0)")
let q = (30pt, 10pt)
let (t, d) = closest-time(p, q)
mp-fig(
  draw(p, pen: pencircle(1.5pt), fill: gray),
  mp-dot(q, pen: pencircle(5pt), fill: blue),
  mp-dot(point-of(p, t), pen: pencircle(5pt), fill: red),
  draw(straight(q, point-of(p, t)), pen:
pencircle(0.6pt)),
  mp-label(text(7pt)[d = #calc.round(d.pt(), digits: 1)],
(60pt, 30pt)),
  pad: 4pt)

```



intersection-times

```
intersection-times(a, b)
```

Tous les croisements de deux chemins, sous forme de couples de temps `(ta, tb)` triés par `ta` (`intersectiontimes` de MetaPost, mais renvoyant **tous** les croisements).

PARAMÈTRES

nom	type	défaut	description
<code>a</code>	<code>chemin</code>	obligatoire	Premier chemin.
<code>b</code>	<code>chemin</code>	obligatoire	Second chemin.

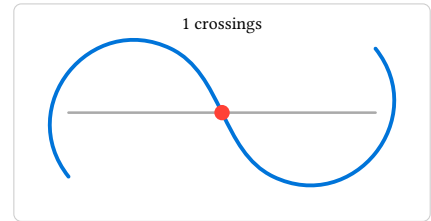
RENVOIE un tableau de couples `(ta, tb)` (vide s'il n'y en a pas).

EXEMPLE

```

let a = mp-path("(0,0)..(40,50)..(80,0)..(120,50)")
let b = mp-path("(0,25)--(120,25)")
let xs = intersection-times(a, b)
mp-fig(
  draw(a, pen: pencircle(1.5pt), fill: blue),
  draw(b, pen: pencircle(1pt), fill: gray),
  ..xs.map(x => mp-dot(point-of(a, x.at(0)),
    pen: pencircle(6pt), fill: red)),
  mp-label(text(7pt)[#xs.len() crossings], (60pt, 60pt)),
  pad: 6pt)

```



intersection-point

```
intersection-point(a, b)
```

Le premier point de croisement de deux chemins (plus petit temps sur `a`), ou `none`.

PARAMÈTRES

nom	type	défaut	description
<code>a</code>	chemin	obligatoire	Premier chemin.
<code>b</code>	chemin	obligatoire	Second chemin.

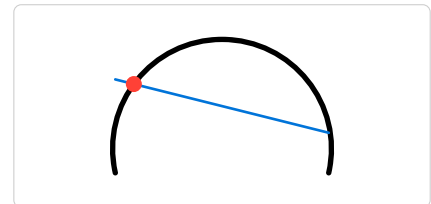
RENVOIE un point, ou `none`.

EXEMPLE

```

let a = mp-path("(0,0)..(40,50)..(80,0)")
let b = mp-path("(0,35)--(80,15)")
let x = intersection-point(a, b)
mp-fig(
  draw(a, pen: pencircle(2pt)),
  draw(b, pen: pencircle(1pt), fill: blue),
  mp-dot(x, pen: pencircle(6pt), fill: red),
  pad: 5pt)

```



self-intersections

```
self-intersections(p)
```

Auto-croisements d'un chemin : couples de temps (`t1`, `t2`) avec `t1 < t2` où le chemin repasse par un même point.

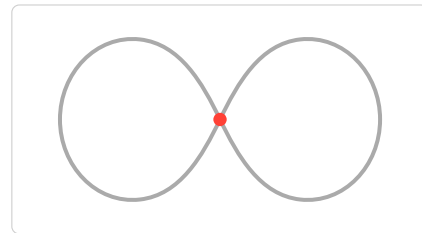
PARAMÈTRES

nom	type	défaut	description
<code>p</code>	chemin	obligatoire	Chemin.

RENVOIE un tableau de couples de temps.

EXEMPLE


```
// une boucle en huit
let p = mp-path("(0,0)..(30,30)..(60,0)..(30,-30)..
(0,0)..(-30,30)..(-60,0)..(-30,-30)..cycle")
let xs = self-intersections(p)
mp-fig(
  draw(p, pen: pencircle(1.5pt), fill: gray),
  ..xs.map(x => mp-dot(point-of(p, x.at(0)),
    pen: pencircle(5pt), fill: red)),
  pad: 5pt)
```



frame-at

```
frame-at(p, t)
```

Repère local du chemin au temps `t` : position et direction de la tangente. À donner à `in-frame` pour placer un dessin le long du chemin.

PARAMÈTRES

nom	type	défaut	description
<code>p</code>	chemin	obligatoire	Chemin.
<code>t</code>	nombre	obligatoire	Temps.

RENVOIE un dictionnaire `(pos: (x, y), angle: angle)`.

EXEMPLE

```
let p = mp-path("(0,0)..(40,40)..(80,0)")
let f = frame-at(p, 1)
[position: (#calc.round(f.pos.at(0).pt()),
#calc.round(f.pos.at(1).pt())) pt \
angle: #calc.round(f.angle.deg())°]
```

position: (40, 40) pt
angle: 0°

in-frame

```
in-frame(q, fr, scale: 1)
```

Place un chemin `q`, dessiné dans un repère **local** (x le long de la tangente, y à gauche), dans le repère `fr` renvoyé par `frame-at`. Idéal pour des pointes de flèche, des perles, des épines, des hachures.

PARAMÈTRES

nom	type	défaut	description
<code>q</code>	chemin	obligatoire	Chemin en coordonnées locales.
<code>fr</code>	dictionnaire	obligatoire	Repère issu de <code>frame-at</code> ou <code>frames-along</code> .
<code>scale</code>	nombre	1	Échelle uniforme appliquée avant la mise en place.

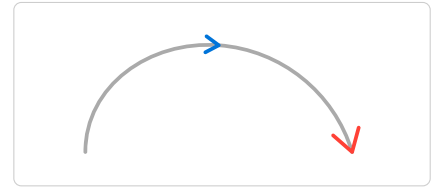
RENVOIE un chemin.

EXEMPLE

```

let p = mp-path("(0,0){up}..(50,40)..(100,0)")
// an arrowhead drawn pointing along +x
// une pointe de flèche dessinée vers +x
let head = polyline((-8pt, 5pt), (0pt, 0pt), (-8pt, -5pt))
let end = frame-at(p, path-length(p))
mp-fig(
  draw(p, pen: pencircle(1.4pt), fill: gray),
  draw(in-frame(head, end), pen: pencircle(1.4pt), fill: red),
  draw(in-frame(head, frame-at(p, 1), scale: 0.6),
    pen: pencircle(1.4pt), fill: blue),
  pad: 5pt)

```



frames-along

```
frames-along(p, count: none, step: none, start: 0pt, end: 0pt)
```

Repères régulièrement espacés le long d'un chemin (en abscisse curviligne) : `count` repères répartis uniformément (sur un chemin fermé, uniformément autour de la boucle), ou un tous les `step`. Les marges optionnelles `start` et `end` sont retirées aux extrémités.

PARAMÈTRES

nom	type	défaut	description
<code>p</code>	chemin	obligatoire	Chemin.
<code>count</code>	entier ou none	none	Nombre de repères.
<code>step</code>	longueur ou none	none	Distance entre repères (alternative à <code>count</code>).
<code>start</code>	longueur	0pt	Longueur d'arc ignorée au début.
<code>end</code>	longueur	0pt	Longueur d'arc ignorée à la fin.

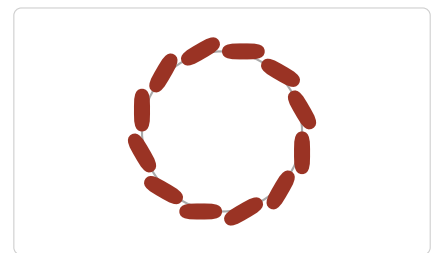
RENVOIE un tableau de repères (`pos:`, `angle:`).

EXEMPLE

```

let ring = mp-path("(0,0)..(30,30)..(60,0)..
(30,-30)..cycle")
// 12 petals around a closed path, pointing outwards
// 12 pétales autour d'un chemin fermé, vers l'extérieur
let petal = mp-path("(0,0)..(8,3)..(16,0)..
(8,-3)..cycle")
mp-fig(
  draw(ring, pen: pencircle(0.8pt), fill: gray),
  ..frames-along(ring, count: 12).map(f =>
    mp-fill(in-frame(petal, f), fill: rgb("#9a3324"))),
  pad: 4pt)

```



Découper et modifier

Ces fonctions renvoient de **nouveaux** chemins (les chemins sont immuables). Celles qui découpent un chemin en morceaux renvoient un tableau de chemins.

subpath

```
subpath(p, t0, t1)
```

Portion du chemin entre les temps `t0` et `t1` (`subpath (t0, t1) of p`). Si `t1 < t0`, la portion est parcourue à rebours. Sur un chemin cyclique, elle peut passer par le départ ; les temps négatifs sont permis.

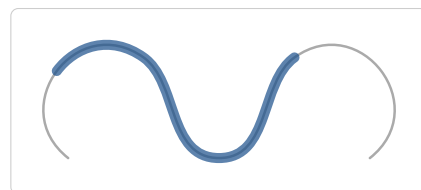
PARAMÈTRES

nom	type	défaut	description
<code>p</code>	chemin	obligatoire	Chemin.
<code>t0</code>	nombre	obligatoire	Temps de départ.
<code>t1</code>	nombre	obligatoire	Temps d'arrivée.

RENVOIE un chemin ouvert.

EXEMPLE

```
let p = mp-path("(0,0)..(30,40)..(60,0)..(90,40)..
(120,0)")
mp-fig(
  draw(p, pen: pencircle(1pt), fill: gray),
  // from the middle of segment 1 to the node 3
  // du milieu du segment 1 au nœud 3
  draw(subpath(p, 0.5, 3), pen: pencircle(4pt),
    fill: rgb("#1a4f8b").transparentize(30%)),
  pad: 5pt)
```



reverse

```
reverse(p)
```

Le même chemin parcouru dans l'autre sens.

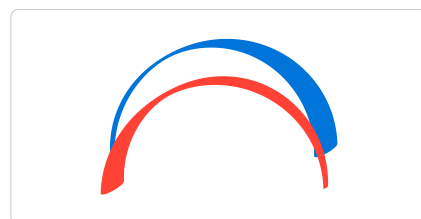
PARAMÈTRES

nom	type	défaut	description
<code>p</code>	chemin	obligatoire	Chemin.

RENVOIE un chemin.

EXEMPLE

```
let p = mp-path("(0,0){up}..(40,40)..(80,0)")
// both are drawn with a pen that thickens along the path
// les deux sont tracés avec une plume qui épaissit le
long du chemin
let pen = nibpen((at: 0%, width: 2pt), (at: 100%, width:
10pt))
let r = shifted(reverse(p), 0pt, -14pt)
mp-fig(
  ..stroke-items(p, pen: pen, fill: blue),
  ..stroke-items(r, pen: pen, fill: red),
  pad: 4pt)
```



shorten

```
shorten(p, start: 0pt, end: 0pt)
```

Retire une longueur au début et/ou à la fin d'un chemin ouvert (par exemple pour arrêter une ligne avant une pointe de flèche).

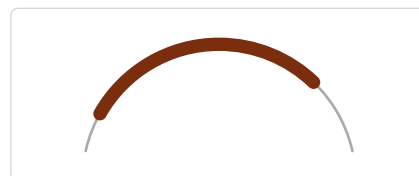
PARAMÈTRES

nom	type	défaut	description
p	chemin	obligatoire	Chemin ouvert.
start	longueur	0pt	Longueur d'arc retirée au début.
end	longueur	0pt	Longueur d'arc retirée à la fin.

RENVOIE un chemin (erreur s'il ne reste rien).

EXEMPLE

```
let p = mp-path("(0,0)..(50,40)..(100,0)")
mp-fig(
  draw(p, pen: pencircle(1pt), fill: gray),
  draw(shorten(p, start: 15pt, end: 30pt),
    pen: pencircle(5pt), fill: rgb("#7a2e0e")),
  pad: 4pt)
```



split-at

```
split-at(p, times)
```

Coupe un chemin aux temps donnés. Un chemin ouvert coupé en n temps donne $n + 1$ morceaux ; un chemin fermé donne n morceaux (le dernier passe par le départ).

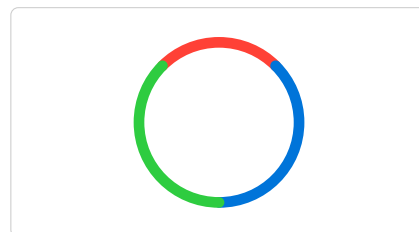
PARAMÈTRES

nom	type	défaut	description
p	chemin	obligatoire	Chemin.
times	tableau de nombres	obligatoire	Temps de coupe.

RENVOIE un tableau de chemins.

EXEMPLE

```
let ring = mp-path("(0,0)..(30,30)..(60,0)..
(30,-30)..cycle")
let cols = (red, blue, green)
let parts = split-at(ring, (0.5, 1.5, 3))
mp-fig(
  ..parts.enumerate().map(((i, q)) => draw(
    q, pen: pencircle(4pt), fill: cols.at(i))),
  pad: 4pt)
```



remove-intervals

```
remove-intervals(p, ivs, min-piece: 0.05pt)
```

Retire des intervalles d'abscisse curviligne d'un chemin et renvoie les morceaux restants. Sur un chemin fermé, les intervalles peuvent passer par le départ ; le morceau à cheval sur la jointure est ressoudé. Base des croisements dessus/dessous.

PARAMÈTRES

nom	type	défaut	description
p	chemin	obligatoire	Chemin.
ivs	tableau de couples	obligatoire	Intervalles $((a, b), \dots)$ d'abscisse curviligne (longueurs).
min-piece	longueur	0.05pt	Les morceaux plus courts sont abandonnés.

RENVOIE un tableau de chemins.

EXEMPLE

```
let p = mp-path("(0,0)..(50,40)..(100,0)..(150,40)")
// remove two holes: [30, 45] and [90, 110]
// retire deux trous : [30, 45] et [90, 110]
let pieces = remove-intervals(p,
  ((30pt, 45pt), (90pt, 110pt)))
mp-fig(
  ..pieces.map(q => draw(q, pen: pencircle(4pt),
    fill: rgb("#1a4f8b"))),
  pad: 4pt)
```



gap-at

```
gap-at(p, times, half)
```

Ouvre un trou de demi-longueur `half` (abscisse curviligne) autour de chacun des temps donnés, et renvoie les morceaux.

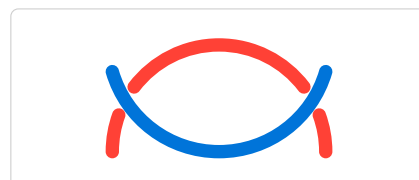
PARAMÈTRES

nom	type	défaut	description
<code>p</code>	chemin	obligatoire	Chemin.
<code>times</code>	tableau de nombres	obligatoire	Temps où couper.
<code>half</code>	longueur ou tableau	obligatoire	Demi-longueur de chaque trou (une valeur, ou une par temps).

RENVOIE un tableau de chemins.

EXEMPLE

```
// two strands crossing: the red one passes under
// deux brins qui se croisent : le rouge passe dessous
let a = mp-path("(0,0)..(40,40)..(80,0)")
let b = mp-path("(0,30)..(40,0)..(80,30)")
let ts = intersection-times(a, b).map(x => x.at(0))
mp-fig(
  draw(b, pen: pencircle(5pt), fill: blue),
  ..gap-at(a, ts, 6pt).map(q =>
    draw(q, pen: pencircle(5pt), fill: red)),
  pad: 4pt)
```



weld

```
weld(paths, tol: 0.05pt)
```

Raccorde les chemins consécutifs d'une liste dès que la fin de l'un est à moins de `tol` du début du suivant.

PARAMÈTRES

nom	type	défaut	description
<code>paths</code>	tableau de chemins	obligatoire	Morceaux, dans l'ordre.
<code>tol</code>	longueur	<code>0.05pt</code>	Distance en dessous de laquelle deux extrémités sont confondues.

RENVOIE un tableau de chemins (moins nombreux si des soudures ont eu lieu).

EXEMPLE

```
let p = mp-path("(0,0)..(50,40)..(100,0)")
let (a, b, c) = split-at(p, (0.7, 1.4))
// a-b-c are three pieces; weld puts them back together
// a-b-c sont trois morceaux ; weld les remet bout à bout
let w = weld((a, b, c))
[pieces: 3 → #w.len()]
```

pieces: 3 → 1

fit-between

```
fit-between(p, a, b)
```

Applique à **p** la similitude (rotation + échelle uniforme + translation) qui envoie son début sur **a** et sa fin sur **b**. Sert à étirer un ornement entre deux points.

PARAMÈTRES

nom	type	défaut	description
p	chemin	obligatoire	Chemin dont les extrémités diffèrent.
a	point	obligatoire	Cible du point de départ.
b	point	obligatoire	Cible du point d'arrivée.

RENVOIE un chemin.

EXEMPLE

```
// a wave drawn once, stretched between several point
pairs
// une vague dessinée une fois, étirée entre plusieurs
paires de points
let wave = mp-path("(0,0)..(10,8)..(20,-8)..(30,0)")
let pairs = (((0pt, 0pt), (60pt, 0pt)), ((0pt, 10pt),
(50pt, 40pt)),
((0pt, -10pt), (30pt, -40pt)))
mp-fig(
  ..pairs.map(((a, b)) => draw(fit-between(wave, a, b),
    pen: pencircle(1.5pt), fill: rgb("#7a2e0e"))),
  pad: 4pt)
```



offset

```
offset(p, d, tol: 0.02pt)
```

Courbe parallèle à la distance signée **d** du chemin (positive = à **gauche** du sens de parcours). Les morceaux sont raccordés par des biseaux droits ; sur un chemin fermé le résultat est fermé.

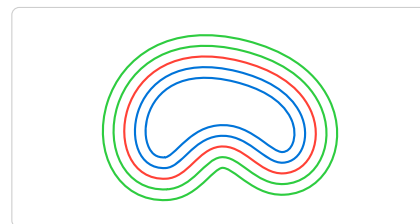
PARAMÈTRES

nom	type	défaut	description
p	chemin	obligatoire	Chemin.
d	longueur	obligatoire	Distance signée.
tol	longueur	0.02pt	Précision de l'approximation.

RENVOIE un chemin.

EXEMPLE

```
let p = mp-path-pts(((0pt, 0pt), (20pt, 12pt), (44pt,
2pt),
(54pt, 26pt), (28pt, 44pt)), cycle: true)
mp-fig(
  ..range(-2, 3).map(k => stroke-items(
    offset(p, k * 4pt), pen: pencircle(0.8pt),
    fill: if k < 0 { green } else if k == 0 { red } else
{ blue })
  ).flatten(),
  pad: 3pt)
```



Transformations

Transformations affines, comme en MetaPost (`shifted`, `scaled`, `rotated`...). Elles agissent sur le **chemin** — à appliquer avant de dessiner. En MetaPost `p scaled 2 shifted (1,1)` se lit de gauche à droite ; ici, imbriquez les appels : `shifted(scaled(p, 2), 1pt, 1pt)`.

shifted

```
shifted(p, dx, dy)
```

Translate le chemin de `(dx, dy)`.

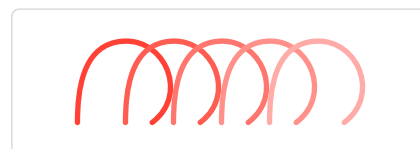
PARAMÈTRES

nom	type	défaut	description
<code>p</code>	<code>chemin</code>	obligatoire	Chemin.
<code>dx</code>	<code>longueur</code>	obligatoire	Décalage horizontal.
<code>dy</code>	<code>longueur</code>	obligatoire	Décalage vertical (y vers le haut).

RENVOIE un chemin.

EXEMPLE

```
let p = mp-path("(0,0){up}..(14,30)..(28,0)")
mp-fig(
  ..range(5).map(i => draw(shifted(p, i * 18pt, 0pt),
    pen: pencircle(2pt), fill: red.lighten(i * 14%))),
  pad: 4pt)
```



scaled

```
scaled(p, s)
```

Met le chemin à l'échelle `s` autour de l'origine.

PARAMÈTRES

nom	type	défaut	description
<code>p</code>	<code>chemin</code>	obligatoire	Chemin.
<code>s</code>	<code>nombre</code>	obligatoire	Facteur d'échelle (négatif = symétrie centrale).

RENVOIE un chemin.

EXEMPLE

```
let p = mp-path("(0,0){up}..(14,30)..(28,0)")
mp-fig(
  ..(0.4, 0.7, 1, 1.5).map(s => draw(scaled(p, s),
    pen: pencircle(1.5pt), fill: blue)),
  pad: 4pt)
```



xscaled

`xscaled(p, s)`

Étire horizontalement seulement (`x` multiplié par `s`).

PARAMÈTRES

nom	type	défaut	description
<code>p</code>	chemin	obligatoire	Chemin.
<code>s</code>	nombre	obligatoire	Facteur horizontal.

RENVOIE un chemin.

EXEMPLE

```
let p = mp-path("(0,0)..(15,15)..(30,0)..
(15,-15)..cycle")
mp-fig(
  draw(p, pen: pencircle(1.5pt), fill: gray),
  draw(shifted(xscaled(p, 2), 80pt, 0pt),
    pen: pencircle(1.5pt), fill: red),
  pad: 4pt)
```



yscaled

`yscaled(p, s)`

Étire verticalement seulement (`y` multiplié par `s`). Un facteur négatif retourne le chemin tête en bas.

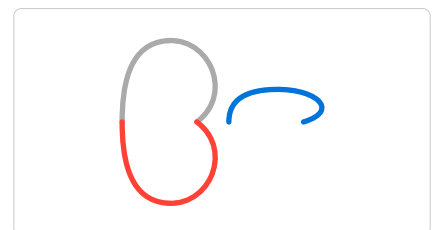
PARAMÈTRES

nom	type	défaut	description
<code>p</code>	chemin	obligatoire	Chemin.
<code>s</code>	nombre	obligatoire	Facteur vertical.

RENVOIE un chemin.

EXEMPLE

```
let p = mp-path("(0,0){up}..(14,30)..(28,0)")
mp-fig(
  draw(p, pen: pencircle(2pt), fill: gray),
  draw(yscaled(p, -1), pen: pencircle(2pt), fill: red),
  draw(shifted(yscaled(p, 0.4), 40pt, 0pt),
    pen: pencircle(2pt), fill: blue),
  pad: 4pt)
```



slanted

`slanted(p, s)`

Cisaillement : chaque point (x, y) devient $(x + s \cdot y, y)$. Pratique pour pencher une écriture.

PARAMÈTRES

nom	type	défaut	description
p	chemin	obligatoire	Chemin.
s	nombre	obligatoire	Inclinaison ($0.25 \approx 14^\circ$).

RENVOIE un chemin.

EXEMPLE

```
let p = mp-path("(0,0)--(0,40)--(25,40)--(25,0)--cycle")
mp-fig(
  ..(0, 0.25, 0.6).enumerate().map(((i, s)) =>
    draw(shifted(slanted(p, s), i * 45pt, 0pt),
      pen: pencircle(2pt), fill: rgb("#1a4f8b")),
    pad: 4pt)
```



rotated

```
rotated(p, a, about: (0pt, 0pt))
```

Fait tourner le chemin dans le sens anti-horaire de l'angle **a** autour du point **about**.

PARAMÈTRES

nom	type	défaut	description
p	chemin	obligatoire	Chemin.
a	angle	obligatoire	Angle (un nombre est lu en degrés).
about	point	(0pt, 0pt)	Centre de rotation.

RENVOIE un chemin.

EXEMPLE

```
let p = mp-path("(10,0){up}..(24,30)..(38,0)")
// seven copies around the origin
// sept copies autour de l'origine
mp-fig(
  ..range(7).map(i => draw(rotated(p, i * 51.4deg),
    pen: broadnib(5pt, t: 1pt, angle: 30deg),
    fill: rgb("#7a2e0e").lighten(i * 9%))),
    pad: 3pt)
```



reflected

```
reflected(p, angle: 90deg)
```

Symétrique par rapport à la droite passant par l'origine de direction **angle** (90deg = axe vertical : gauche ↔ droite ; 0deg = axe horizontal).

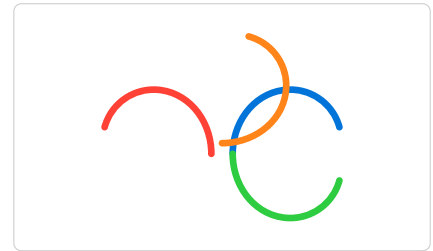
PARAMÈTRES

nom	type	défaut	description
p	chemin	obligatoire	Chemin.
angle	angle	90deg	Direction de l'axe du miroir.

RENVOIE un chemin.

EXEMPLE

```
let p = mp-path("(4,0){up}..(24,24)..(44,10)")
let pen = pencircle(2.5pt)
mp-fig(
  draw(p, pen: pen, fill: blue),
  draw(reflected(p), pen: pen, fill: red),
  draw(reflected(p, angle: 0deg), pen: pen, fill: green),
  draw(reflected(p, angle: 45deg), pen: pen, fill:
orange),
  pad: 4pt)
```



transformed

```
transformed(p, a, b, c, d, e: 0pt, f: 0pt)
```

Transformation affine générale : $(x, y) \mapsto (a \cdot x + c \cdot y + e, b \cdot x + d \cdot y + f)$. Toutes les autres transformations en sont des cas particuliers (rotated : $a = d = \cos \theta$, $b = \sin \theta$, $c = -\sin \theta$).

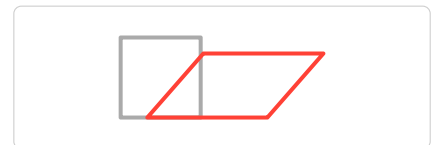
PARAMÈTRES

nom	type	défaut	description
p	chemin	obligatoire	Chemin.
a, b, c, d	nombre	obligatoire	Coefficients de la matrice 2×2 .
e	longueur	0pt	Translation horizontale.
f	longueur	0pt	Translation verticale.

RENVOIE un chemin.

EXEMPLE

```
let sq = mp-path("(0,0)--(30,0)--(30,30)--(0,30)--cycle")
// a unit square sheared and stretched into a
parallelogram
// un carré cisailé et étiré en parallélogramme
let para = transformed(sq, 1.5, 0, 0.7, 0.8, e: 10pt, f:
0pt)
mp-fig(
  draw(sq, pen: pencircle(1.5pt), fill: gray),
  draw(para, pen: pencircle(1.5pt), fill: red),
  pad: 4pt)
```



Plumes

Une **plume** est un simple dictionnaire décrivant la forme qu'on fait glisser le long d'un chemin. Les plumes fixes (`penellipse`, `pencircle`, `broadnib`, `penpoly`, `pensquare`, `penrazor`) ont une forme constante et fonctionnent avec `draw` et `stroke-items` ; `nibpen` construit une plume calligraphique qui peut varier le long du chemin et donne accès à `dash`, `pressure` et `follow` dans `stroke-items`.

Plumes fixes

`penellipse`

```
penellipse(w, h, angle: 0deg)
```

Plume elliptique, comme `pencircle xscaled w yscaled h rotated angle` de MetaPost. Le trait est le plus épais là où le chemin est perpendiculaire au grand axe.

PARAMÈTRES

nom	type	défaut	description
<code>w</code>	<code>longueur</code>	obligatoire	Largeur totale, selon l'axe tourné de <code>angle</code> .
<code>h</code>	<code>longueur</code>	obligatoire	Hauteur totale, perpendiculaire à cet axe.
<code>angle</code>	<code>angle</code>	<code>0deg</code>	Orientation de l'axe <code>w</code> (anti-horaire).

RENVOIE une plume.

EXEMPLE

```
let p = mp-path("(0,0){up}..(15,28)..(30,0)..(45,28)")
let pens = (
  penellipse(10pt, 3pt),
  penellipse(10pt, 3pt, angle: 45deg),
  penellipse(3pt, 10pt),
)
grid(columns: 3, gutter: 6pt,
  ..pens.map(pn => nib-stroke(p, pen: pn, pad: 4pt)))
```



`pencircle`

```
pencircle(d)
```

Plume circulaire de diamètre `d` : un trait de largeur constante `d`.

PARAMÈTRES

nom	type	défaut	description
<code>d</code>	<code>longueur</code>	obligatoire	Diamètre.

RENVOIE une plume.

EXEMPLE

```
let p = mp-path("(0,0)..(30,30)..(60,0)")
stack(dir: ltr, spacing: 8pt,
  ..(1pt, 3pt, 6pt, 10pt).map(d =>
    nib-stroke(p, pen: pencircle(d), pad: 2pt)))
```



broadnib

```
broadnib(w, t: auto, angle: 30deg)
```

Plume plate de calligraphe : une ellipse aplatie. Traits épais perpendiculaires à la plume, déliés parallèles à elle.

PARAMÈTRES

nom	type	défaut	description
w	longueur	obligatoire	Largeur de la plume.
t	longueur ou auto	auto	Épaisseur de la plume (auto = w / 5).
angle	angle	30deg	Inclinaison de la plume, anti-horaire depuis l'horizontale.

RENVOIE une plume.

EXEMPLE

```
// the letter "n" with three nib angles
// la lettre « n » avec trois angles de plume
let n = mp-path("(0,0){up}..(0,24)..(2,16){up}..(14,26)
{right}..(24,14){down}..(24,0)")
stack(dir: ltr, spacing: 10pt,
  ..(0deg, 30deg, 60deg).map(a =>
    nib-stroke(n, pen: broadnib(6pt, angle: a), pad:
      3pt)))
```



penpoly

```
penpoly(pts)
```

Plume polygonale convexe. Les points sont donnés autour de l'origine ; le trait est l'union exacte du polygone déplacé le long du chemin.

PARAMÈTRES

nom	type	défaut	description
pts	tableau de points	obligatoire	Sommets d'un polygone convexe (longueurs).

RENVOIE une plume.

EXEMPLE

```
let p = mp-path("(0,0)..(25,30)..(50,0)..(75,30)")
let tri = penpoly((-6pt, -4pt), (6pt, -4pt), (0pt,
7pt))
nib-stroke(p, pen: tri, pad: 4pt)
```



pensquare

```
pensquare(s, angle: 0deg)
```

Plume carrée de côté s, tournée de angle.

PARAMÈTRES

nom	type	défaut	description
s	longueur	obligatoire	Côté du carré.
angle	angle	0deg	Rotation (anti-horaire).

RENVIOIE une plume.

EXEMPLE

```
let p = mp-path("(0,0)..(25,30)..(50,0)..(75,30)")
stack(dir: ltr, spacing: 8pt,
  ..(0deg, 20deg, 45deg).map(a =>
    nib-stroke(p, pen: pensquare(5pt, angle: a), pad:
      3pt)))
```



penrazor

```
penrazor(len, angle: 0deg)
```

Plume rasoir : un segment de longueur `len` à l'angle `angle`. Donne des traits francs et plats dont la largeur est nulle quand le chemin suit le rasoir.

PARAMÈTRES

nom	type	défaut	description
<code>len</code>	<code>longueur</code>	obligatoire	Longueur du segment.
<code>angle</code>	<code>angle</code>	<code>0deg</code>	Direction du segment.

RENVIOIE une plume.

EXEMPLE

```
let p = mp-path("(0,0)..(25,30)..(50,0)..(75,30)")
stack(dir: ltr, spacing: 8pt,
  ..(0deg, 45deg, 90deg).map(a =>
    nib-stroke(p, pen: penrazor(10pt, angle: a), pad:
      3pt)))
```



La plume calligraphique et ses options

nibpen

```
nibpen(..stops, width: 1pt, thinness: 25%, minor-width: none, angle: 30deg, follow:
false)
```

Plume plate qui peut **varier le long du chemin**. Sans arrêt, ses paramètres sont constants. Un **arrêt** est un dictionnaire (`at:`, `width:`, `thinness:`, `minor-width:`, `angle:`) ; `at` est une proportion de la longueur d'arc (50%) ou une longueur d'arc absolue (2cm). Les clés absentes d'un arrêt prennent les valeurs données à `nibpen`. Les paramètres sont interpolés linéairement entre les arrêts. Avec `follow: true`, l'angle est mesuré depuis la tangente du chemin : la plume tourne avec le trait.

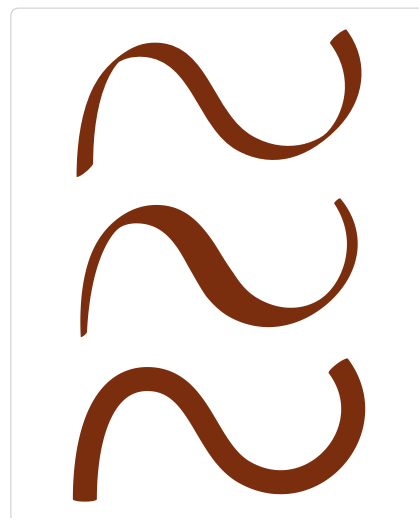
PARAMÈTRES

nom	type	défaut	description
<code>..stops</code>	<code>dictionaries</code>	<code>()</code>	Arrêts optionnels (<code>at:</code> , <code>width:</code> , <code>thinness:</code> , <code>minor-width:</code> , <code>angle:</code>).
<code>width</code>	<code>longueur</code>	<code>1pt</code>	Grand axe par défaut (largeur totale de la plume).
<code>thinness</code>	<code>proportion</code>	<code>25%</code>	Petit axe en proportion de <code>width</code> .
<code>minor-width</code>	<code>longueur ou none</code>	<code>none</code>	Petit axe en longueur absolue (remplace <code>thinness</code>).
<code>angle</code>	<code>angle</code>	<code>30deg</code>	Angle de plume par défaut.
<code>follow</code>	<code>booléen</code>	<code>false</code>	Angle relatif à la tangente au lieu d'être fixe.

RENVOIE une plume variable (à utiliser avec `stroke-items` / `nib-stroke`, pas `draw`).

EXEMPLE

```
let p = mp-path("(0,0){up}..(25,45)..(60,10)..(95,50)")
let ink = rgb("#7a2e0e")
// 1. constant · 2. three stops · 3. follow the path
// 1. constante · 2. trois arrêts · 3. suit le chemin
let a = nibpen(width: 8pt, thinness: 20%, angle: 40deg)
let b = nibpen(
  (at: 0%, width: 3pt), (at: 50%, width: 14pt, thinness:
40%),
  (at: 100%, width: 3pt), angle: 40deg)
let c = nibpen(width: 9pt, thinness: 20%, angle: 90deg,
follow: true)
stack(dir: ttb, spacing: 6pt,
  ..(a, b, c).map(pn => nib-stroke(p, pen: pn, fill:
ink)))
```



dashes

```
dashes(..lengths, offset: 0pt, jitter: 0pt, seed: 0)
```

Motif de pointillés pour `stroke-items(dash: ...)` : longueurs alternées de trait et de vide le long du chemin. `jitter` fait varier chaque longueur au hasard (de façon déterministe, à partir de `seed`), ce qui donne un aspect dessiné à la main.

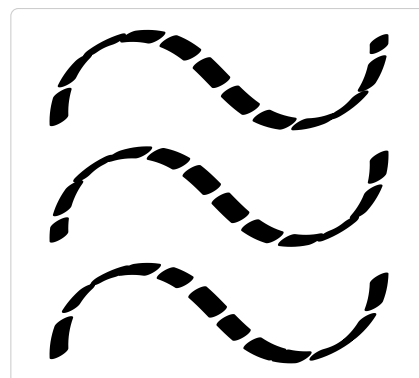
PARAMÈTRES

nom	type	défaut	description
<code>..lengths</code>	<code>lengths</code>	obligatoire	Trait, vide, trait, vide... (répétés cycliquement).
<code>offset</code>	<code>longueur</code>	<code>0pt</code>	Début du motif.
<code>jitter</code>	<code>longueur</code>	<code>0pt</code>	Variation aléatoire $\pm$ <code>jitter</code> de chaque longueur.
<code>seed</code>	<code>entier</code>	<code>0</code>	Graine de la suite aléatoire.

RENVOIE une spécification de pointillés.

EXEMPLE

```
let p = mp-path("(0,0)..(40,30)..(80,0)..(120,30)")
let pen = nibpen(width: 8pt, thinness: 30%, angle: 30deg)
stack(dir: ttb, spacing: 4pt,
  nib-stroke(p, pen: pen, dash: dashes(10pt, 5pt)),
  nib-stroke(p, pen: pen, dash: dashes(10pt, 5pt, offset:
5pt)),
  nib-stroke(p, pen: pen,
    dash: dashes(10pt, 4pt, jitter: 3pt, seed: 7)))
```



pressure

```
pressure(minimum-width: 0.2pt, period: 2pt, seed: 0)
```

Simule une plume qui se soulève : là où le petit axe de la plume passe sous `minimum-width`, le trait se fragmente avec une probabilité croissante quand la plume s'amincit (un tirage par `period` de longueur d'arc). À utiliser avec une plume qui s'amincit.

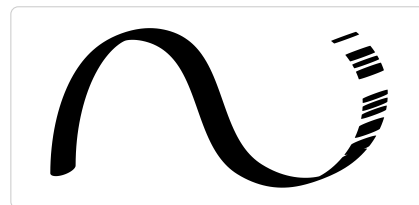
PARAMÈTRES

nom	type	défaut	description
<code>minimum-width</code>	longueur	<code>0.2pt</code>	Petit axe en dessous duquel le trait commence à se fragmenter.
<code>period</code>	longueur	<code>2pt</code>	Longueur de chaque tirage.
<code>seed</code>	entier	<code>0</code>	Graine de la suite aléatoire.

RENVOIE une spécification de pression.

EXEMPLE

```
let p = mp-path("(0,0){up}..(35,50)..(70,0)..(105,50)")
// the pen thins from 40% to 2% of its width
// la plume s'amincit de 40 % à 2 % de sa largeur
let pen = nibpen((at: 0%, width: 10pt, thinness: 40%),
  (at: 100%, width: 10pt, thinness: 2%), angle: 20deg)
nib-stroke(p, pen: pen,
  pressure: pressure(minimum-width: 1.2pt, period: 1.5pt,
    seed: 3))
```



Dessiner

Les fonctions de dessin renvoient des **objets dessinables** : des objets opaques qu'on confie à `mp-fig`, qui les dispose dans un repère y-haut et renvoie une `box` Typst ordinaire. `nib-stroke` est le raccourci pour un seul trait calligraphique dans sa propre boîte.

Traits

draw

```
draw(p, pen: none, pens: none, fill: black, tol: 0.01pt, fit: true, exact: false,
outline: none)
```

Trace le chemin `p` avec une plume, comme `pickup pen; draw p` de MetaPost. Le résultat est la **forme remplie** exacte balayée par la plume. Donnez soit une plume fixe (`pen`), soit, pour une plume variable de bas niveau, `pens` : un triplet `(w, h, angle)` par nœud du chemin. Pour `nibpen`, pointillés, pression... utilisez `stroke-items`.

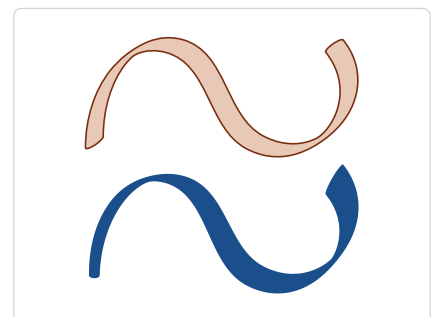
PARAMÈTRES

nom	type	défaut	description
<code>p</code>	<code>chemin</code>	obligatoire	Chemin à tracer.
<code>pen</code>	<code>plume</code>	<code>none</code>	Plume fixe (<code>penellipse</code> , <code>pencircle</code> , <code>broadnib</code> , <code>penpoly</code> , <code>pensquare</code> , <code>penrazor</code>).
<code>pens</code>	<code>tableau ou fonction</code>	<code>none</code>	Plume variable : un <code>(w, h, angle)</code> par nœud (<code>segments + 1</code> , ou <code>segments</code> sur un chemin fermé), ou une fonction de l'indice du nœud.
<code>fill</code>	<code>couleur</code>	<code>black</code>	Remplissage de la forme.
<code>tol</code>	<code>longueur</code>	<code>0.01pt</code>	Tolérance géométrique de l'enveloppe.
<code>fit</code>	<code>booléen</code>	<code>true</code>	Ajuste des courbes de Bézier au contour des plumes variables/polygonales (moins de nœuds).
<code>exact</code>	<code>booléen</code>	<code>false</code>	Force le balayage par union d'enveloppes convexes au lieu de l'enveloppe exacte.
<code>outline</code>	<code>trait ou none</code>	<code>none</code>	Trait Typst dessiné autour de la forme.

RENVOIE un objet dessinable.

EXEMPLE

```
let p = mp-path("(0,0){up}..(30,36)..(60,0)..(90,36)")
mp-fig(
  // fixed pen with a Typst outline
  // plume fixe avec un contour Typst
  draw(p, pen: broadnib(8pt, t: 2pt, angle: 35deg),
    fill: rgb("#e8c9b8"), outline: 0.6pt +
    rgb("#7a2e0e")),
  // variable pen: one (w, h, angle) per node (4 nodes)
  // plume variable : un (w, h, angle) par nœud (4 nœuds)
  draw(shifted(p, 0pt, -50pt),
    pens: i => (4pt + i * 3pt, 2pt, 20deg * i),
    fill: rgb("#1a4f8b")),
  pad: 4pt)
```



stroke-items


```
stroke-items(p, pen: none, pens: none, fill: black, dash: none, pressure: none, follow:
auto, refine: auto, overlap: "union", layer: 4pt, outline: none, tol: 0.01pt, exact:
false, fit: true, debug: false)
```

Le trait calligraphique tout-en-un. Renvoie un **tableau** d'objets dessinables (à étaler dans `mp-fig` avec `..`). Accepte tous les types de plume ; les fonctions propres à `nibpen` exigent une plume elliptique : pointillés, pression, `overlap`: "layered" et `debug`.

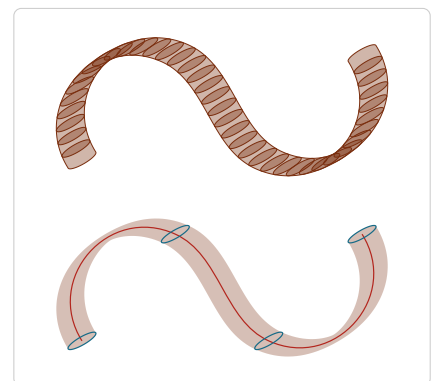
PARAMÈTRES

nom	type	défaut	description
<code>p</code>	chemin	obligatoire	Chemin à tracer.
<code>pen</code>	plume	<code>none</code>	Toute plume, y compris <code>nibpen(...)</code> .
<code>pens</code>	tableau ou fonction	<code>none</code>	Plumes par nœud (comme dans <code>draw</code>) ; incompatible avec <code>dash</code> , <code>pressure</code> , <code>layered</code> et <code>debug</code> .
<code>fill</code>	couleur	<code>black</code>	Remplissage des formes (les couleurs translucides fonctionnent avec <code>layered</code>).
<code>dash</code>	dashes	<code>none</code>	Motif de pointillés issu de <code>dashes(...)</code> .
<code>pressure</code>	pressure	<code>none</code>	Soulèvement de plume issu de <code>pressure(...)</code> .
<code>follow</code>	booléen ou auto	<code>auto</code>	Remplace <code>nibpen(follow:)</code> .
<code>refine</code>	entier ou auto	<code>auto</code>	Subdivision de chaque segment pour les plumes variables (<code>auto</code> = 4 si la plume varie, sinon 1).
<code>overlap</code>	chaîne	<code>"union"</code>	<code>"union"</code> : une seule forme fusionnée ; <code>"layered"</code> : morceaux de longueur <code>layer</code> empilés comme formes séparées.
<code>layer</code>	longueur	<code>4pt</code>	Longueur d'un morceau en mode <code>layered</code> .
<code>outline</code>	trait ou none	<code>none</code>	Trait Typst autour des formes.
<code>tol, exact, fit</code>	—	<code>0.01pt, false, true</code>	Comme dans <code>draw</code> .
<code>debug</code>	booléen	<code>false</code>	Dessine aussi le squelette (rouge) et quelques positions de la plume (cyan).

RENVOIE un tableau d'objets dessinables.

EXEMPLE

```
let p = mp-path("(0,0)..(35,40)..(70,0)..(105,40)")
let ink = rgb("#7a2e0e")
let pen = nibpen(width: 12pt, thinness: 25%, angle: 30deg)
stack(dir: ttb, spacing: 8pt,
// layered, translucent, with outline
// empilé, translucide, avec contour
mp-fig(..stroke-items(p, pen: pen, overlap: "layered",
layer: 5pt, fill: ink.transparentize(65%),
outline: 0.3pt + ink), pad: 4pt),
// construction view
// vue de construction
mp-fig(..stroke-items(p, pen: pen, debug: true,
fill: ink.transparentize(70%)), pad: 4pt))
```



nib-stroke

```
nib-stroke(p, pad: 1pt, ..args)
```

Comme `stroke-items`, mais renvoie directement une boîte prête à l'emploi (un `mp-fig` ajusté au trait plus `pad`). Tous les autres arguments sont transmis à `stroke-items`.

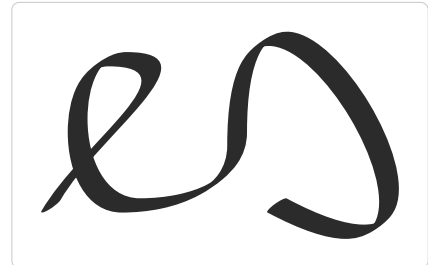
PARAMÈTRES

nom	type	défaut	description
<code>p</code>	<code>chemin</code>	obligatoire	Chemin à tracer.
<code>pad</code>	<code>longueur</code>	<code>1pt</code>	Marge autour du trait.
<code>..args</code>	<code>—</code>	obligatoire	Tout argument de <code>stroke-items</code> (<code>pen</code> , <code>fill</code> , <code>dash</code> , ...).

RENVOIE une boîte (ligne de base sur `y = 0` quand le trait la traverse).

EXEMPLE

```
let p = mp-path("(0,0){dir 60}..(25,55){left}..(10,30)
{down}..(30,0){right}..(70,25){up}..(85,0)")
nib-stroke(p, pad: 4pt, fill: rgb("#2b2b2b"),
pen: broadnib(9pt, t: 1.8pt, angle: 35deg))
```



Remplissages, points, étiquettes

mp-fill

```
mp-fill(p, fill: black)
```

Remplit l'aire délimitée par un chemin fermé (`fill p`).

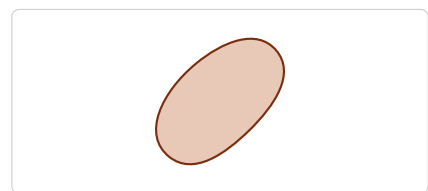
PARAMÈTRES

nom	type	défaut	description
<code>p</code>	<code>chemin</code>	obligatoire	Chemin fermé (un chemin ouvert est fermé par une droite).
<code>fill</code>	<code>couleur</code>	<code>black</code>	Couleur de remplissage.

RENVOIE un objet dessinaable.

EXEMPLE

```
let blob = mp-path("(0,0)..(30,10)..(40,40)..
(10,35)..cycle")
mp-fig(
  mp-fill(blob, fill: rgb("#e8c9b8")),
  mp-skeleton(blob, stroke: 0.8pt + rgb("#7a2e0e")),
  pad: 4pt)
```



mp-dot

```
mp-dot(pos, pen: pencircle(2pt), fill: black)
```

Un point : la forme de la plume dessinée en une position (`drawdot`).

PARAMÈTRES

nom	type	défaut	description
<code>pos</code>	<code>point</code>	obligatoire	Position (<code>x</code> , <code>y</code>).
<code>pen</code>	<code>plume</code>	<code>pencircle(2pt)</code>	Plume dont la forme est le point (rond, elliptique, carré...).
<code>fill</code>	<code>couleur</code>	<code>black</code>	Couleur.

RENVOIE un objet dessinaable.

EXEMPLE

```
mp-fig(
  mp-dot((0pt, 0pt), pen: pencircle(8pt), fill: red),
  mp-dot((20pt, 0pt), pen: pensquare(7pt, angle: 20deg),
  fill: blue),
  mp-dot((40pt, 0pt), pen: penellipse(12pt, 5pt, angle:
30deg)),
  mp-dot((60pt, 0pt), pen: penpoly((( -5pt, -4pt), (5pt,
-4pt), (0pt, 6pt))), fill: green),
  pad: 4pt)
```



mp-label

```
mp-label(body, pos, align: center + horizon)
```

Un contenu Typst placé en une position, aligné comme `label.top / .lft ...` de MetaPost. `align` indique quel point du contenu est posé en `pos`. Les étiquettes ne comptent pas dans la boîte englobante automatique de `mp-fig` (utilisez `pad`).

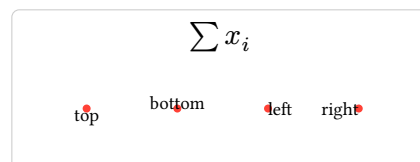
PARAMÈTRES

nom	type	défaut	description
<code>body</code>	<code>contenu</code>	obligatoire	Tout contenu : texte, maths, image...
<code>pos</code>	<code>point</code>	obligatoire	Position <code>(x, y)</code> .
<code>align</code>	<code>alignement</code>	<code>center + horizon</code>	Ancrage du contenu : <code>top</code> , <code>bottom</code> , <code>left</code> , <code>right</code> , <code>center</code> , <code>horizon</code> et leurs combinaisons.

RENVOIE un objet dessinaable.

EXEMPLE

```
// each dot is the anchor `pos`; the label hangs from it
// chaque point est l'ancre `pos` ; l'étiquette s'y
accroche
let anchors = (top, bottom, left, right)
mp-fig(
  ..anchors.enumerate().map((i, a)) => {
    let o = (i * 34pt, 0pt)
    mp-group(
      mp-dot(o, pen: pencircle(3pt), fill: red),
      mp-label(text(7pt, repr(a)), o, align: a))
  }),
  mp-label(text(11pt)[$sum x_i$], (50pt, 28pt)),
  width: 140pt, height: 46pt, origin: (20pt, 16pt))
```



mp-skeleton

```
mp-skeleton(p, stroke: 0.4pt + red)
```

Dessine le squelette nu d'un chemin avec un `stroke` Typst (pratique pour vérifier un chemin avant de choisir une plume).

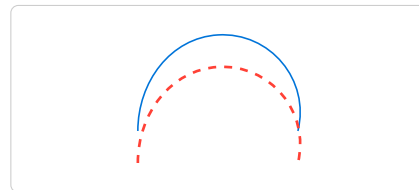
PARAMÈTRES

nom	type	défaut	description
<code>p</code>	<code>chemin</code>	obligatoire	Chemin.
<code>stroke</code>	<code>trait</code>	<code>0.4pt + red</code>	Tout trait Typst (pointillé, coloré...).

RENVOIE un objet dessinable.

EXEMPLE

```
let p = mp-path("(0,0){up}..(30,36)..(60,0)")
mp-fig(
  mp-skeleton(p, stroke: 0.6pt + blue),
  mp-skeleton(shifted(p, 0pt, -12pt),
    stroke: (paint: red, thickness: 1pt, dash:
"dashed")),
  pad: 4pt)
```



mp-group

```
mp-group(..items)
```

Regroupe plusieurs objets dessinables en un seul (pour qu'une fonction renvoie un objet unique). `mp-fig` accepte aussi directement des tableaux d'objets, le regroupement est donc facultatif.

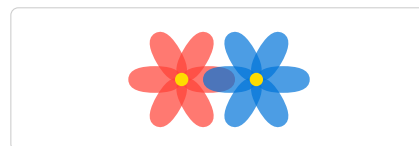
PARAMÈTRES

nom	type	défaut	description
<code>..items</code>	<code>drawables</code>	obligatoire	Objets à regrouper (les tableaux sont aplatis).

RENVOIE un objet dessinable.

EXEMPLE

```
// a reusable "flower": one object made of several
// une « fleur » réutilisable : un objet fait de
// plusieurs
let petal = mp-path("(0,0)..(10,5)..(20,0)..
(10,-5)..cycle")
let flower(c, dx) = mp-group(
  ..range(6).map(i => mp-fill(
    shifted(rotated(petal, i * 60deg), dx, 0pt),
    fill: c.transparentize(30%))),
  mp-dot((dx, 0pt), pen: pencircle(5pt), fill: yellow))
mp-fig(flower(red, 0pt), flower(blue, 28pt), pad: 2pt)
```



Figures

mp-fig

```
mp-fig(..items, width: auto, height: auto, origin: (0pt, 0pt), pad: 1pt, baseline: auto,
grid: none, frame: none, clip: false)
```

Rassemble des objets dessinables dans un repère y-haut (comme `beginfig ... endfig`) et renvoie une `box`. Avec des tailles `auto`, la boîte s'ajuste au contenu plus `pad` ; avec des tailles explicites, `origin` est la position de l'origine comptée depuis le coin bas-gauche. La boîte peut s'insérer dans le texte courant : sa ligne de base est la droite `y = 0` sauf si `baseline` est donné.

PARAMÈTRES

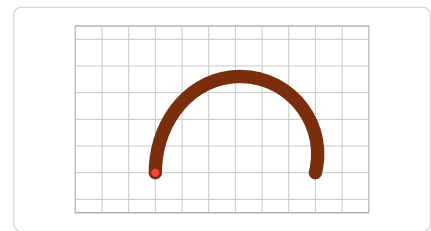
nom	type	défaut	description
<code>..items</code>	<code>drawables</code>	obligatoire	Objets à dessiner, dans l'ordre (les derniers dessus). Les tableaux sont aplatis : <code>..stroke-items(...)</code> ou <code>stroke-items(...)</code> conviennent.
<code>width</code>	<code>longueur ou auto</code>	<code>auto</code>	Largeur de la boîte (<code>auto</code> = ajustée au contenu).

nom	type	défaut	description
<code>height</code>	longueur ou auto	<code>auto</code>	Hauteur de la boîte.
<code>origin</code>	point	<code>(0pt, 0pt)</code>	Position de l'origine depuis le coin bas-gauche (tailles explicites seulement).
<code>pad</code>	longueur	<code>1pt</code>	Marge ajoutée autour du contenu quand une taille est <code>auto</code> .
<code>baseline</code>	longueur ou auto	<code>auto</code>	Décalage de la ligne de base depuis le bas de la boîte.
<code>grid</code>	longueur ou none	<code>none</code>	Trace une grille légère de cet espacement (pour la mise au point).
<code>frame</code>	trait ou none	<code>none</code>	Trait de bordure autour de la boîte.
<code>clip</code>	booléen	<code>false</code>	Coupe ce qui dépasse de la boîte.

RENVOIE une `box`.

EXEMPLE

```
let p = mp-path("(0,0){up}..(30,36)..(60,0)")
// explicit size, origin in the middle, grid, frame
// taille explicite, origine au milieu, grille, cadre
mp-fig(
  draw(p, pen: pencircle(5pt), fill: rgb("#7a2e0e")),
  mp-dot((0pt, 0pt), pen: pencircle(3pt), fill: red),
  width: 110pt, height: 70pt, origin: (30pt, 15pt),
  grid: 10pt, frame: 0.5pt + gray, clip: true)
```



Nœuds, ornements et décorations

Cette couche (`art.typ`, réexportée par le paquet) transforme le moteur géométrique en ornements prêts à l'emploi. Les idées suivent les paquets LaTeX `spath3`, `knots` et `calligraphy` ; l'implémentation est indépendante. Comme le reste, ces fonctions renvoient des tableaux d'objets dessinables (à étaler dans `mp-fig` avec `..`).

Nœuds et entrelacs

knot

```
knot(strands, style: "gap", rule: "alternate", width: 4pt, fill: black, outline: none,
outline-fill: black, gap: auto, flips: (), draft: false, draft-size: 7pt)
```

Entrelacs : nœuds, liens, tresses, motifs celtiques. Donnez un chemin ou un tableau de chemins (ouverts ou fermés). Chaque croisement — entre brins ou à l'intérieur d'un brin — est détecté automatiquement et numéroté dans l'ordre de première rencontre. `style: "gap"` interrompt le brin du dessous (diagramme de nœud classique) ; `style: "weave"` dessine chaque brin comme un ruban avec un contour et redessine le brin du dessus à chaque croisement (entrelacs celtique ; exige `outline`).

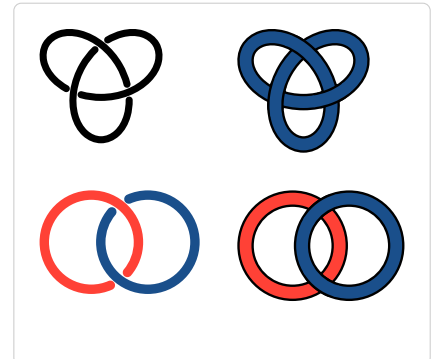
PARAMÈTRES

nom	type	défaut	description
<code>strands</code>	chemin ou tableau de chemins	obligatoire	Les brins.
<code>style</code>	chaîne	"gap"	"gap" ou "weave" (sans <code>outline</code> , "weave" retombe sur "gap").
<code>rule</code>	chaîne	"alternate"	"alternate" : diagramme alterné (dessus, dessous, dessus... le long de chaque brin). "first" : le brin d'indice le plus bas passe dessus (dans un même brin, le dernier passage).
<code>width</code>	longueur ou tableau	4pt	Largeur des brins (une valeur, ou une par brin).
<code>fill</code>	couleur ou tableau	black	Couleur des brins (une valeur, ou une par brin).
<code>outline</code>	longueur ou none	none	Largeur du contour de chaque côté d'un ruban.
<code>outline-fill</code>	couleur ou tableau	black	Couleur du contour.
<code>gap</code>	longueur ou auto	auto	Dégagement de chaque côté d'un trou, style "gap" (<code>auto</code> = $0,6 \times$ largeur).
<code>flips</code>	tableau de integers	()	Numéros des croisements à inverser.
<code>draft</code>	booléen	false	Affiche les squelettes et les numéros de croisement (pour choisir <code>flips</code>).
<code>draft-size</code>	longueur	7pt	Taille des numéros en mode brouillon.

RENVOIE un tableau d'objets dessinables.

EXEMPLE

```
// a trefoil: 24 points of a parametric curve
// un trèfle : 24 points d'une courbe paramétrique
let pt(i) = {
  let t = i * 15deg
  (9pt * (calc.sin(t) + 2 * calc.sin(2 * t)),
   9pt * (calc.cos(t) - 2 * calc.cos(2 * t)))
}
let tre = mp-path-pts(range(24).map(pt), cycle: true)
// a link of two circles
// un lien de deux cercles
let c1 = mp-path("(0,0)..(20,20)..(40,0)..
(20,-20)..cycle")
let c2 = shifted(c1, 24pt, 0pt)
let blue = rgb("#1a4f8b")
grid(columns: 2, gutter: 10pt,
  mp-fig(..knot(tre, width: 3pt), pad: 3pt),
  mp-fig(..knot(tre, style: "weave", width: 5pt, fill:
blue,
  outline: 1pt), pad: 3pt),
  mp-fig(..knot((c1, c2), width: 4pt, fill: (red, blue)),
  pad: 3pt),
  mp-fig(..knot((c1, c2), style: "weave", width: 5pt,
  fill: (red, blue), outline: 1pt, flips: (1,)), pad:
3pt))
```



crossings

`crossings(strands)`

Les données brutes derrière `knot` : tous les croisements d'un ensemble de brins, y compris les auto-intersections de chaque brin.

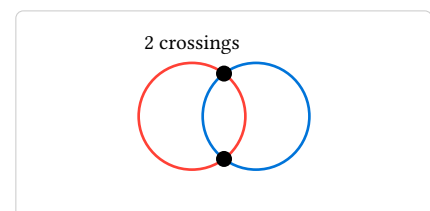
PARAMÈTRES

nom	type	défaut	description
strands	chemin ou tableau de chemins	obligatoire	Les brins.

RENVOIE un tableau de `(a: (brin, t), b: (brin, t))` : indice du brin et temps des deux passages.

EXEMPLE

```
let c1 = mp-path("(0,0)..(20,20)..(40,0)..
(20,-20)..cycle")
let c2 = shifted(c1, 24pt, 0pt)
let cs = crossings((c1, c2))
mp-fig(
  draw(c1, pen: pencircle(1pt), fill: red),
  draw(c2, pen: pencircle(1pt), fill: blue),
  // the point where the first strand passes through each
  crossing
  // le point où le premier brin traverse chaque
  croisement
  ..cs.map(c => mp-dot(point-of(c1, c.a.at(1)),
    pen: pencircle(6pt), fill: black)),
  mp-label(text(8pt)[#cs.len() crossings], (20pt, 28pt)),
  pad: 12pt)
```



Décorations calligraphiques

copperplate

```
copperplate(p, light: 0.4pt, heavy: 2.4pt, slant: 90deg, sharpness: 1.0, taper: "both", taper-length: 6pt, step: 1.5pt, fill: black, outline: none)
```

Trait à la **plume pointue** (copperplate, ronde) : un délié dans les montées, un plein dans les descentes. La largeur suit la direction du chemin par rapport à l'axe d'ombre `slant`. Renvoie un tableau d'objets dessinables.

PARAMÈTRES

nom	type	défaut	description
<code>p</code>	chemin	obligatoire	Chemin du trait.
<code>light</code>	longueur	0.4pt	Largeur du délié.
<code>heavy</code>	longueur	2.4pt	Largeur du plein.
<code>slant</code>	angle	90deg	Direction de l'axe d'ombre (90deg = vertical ; moins = italique).
<code>sharpness</code>	nombre	1.0	> 1 limite le plein aux traits plus proches de l'axe.
<code>taper</code>	chaîne ou none	"both"	Attaque/sortie en délié : <code>none</code> , <code>"start"</code> , <code>"end"</code> , <code>"both"</code> .
<code>taper-length</code>	longueur	6pt	Longueur des extrémités effilées.
<code>step</code>	longueur	1.5pt	Pas d'échantillonnage (plus petit = plus lisse, plus lent).
<code>fill</code>	couleur	black	Couleur de l'encre.
<code>outline</code>	trait ou none	none	Trait Typst autour de la forme.

RENVOIE un tableau d'objets dessinables.

EXEMPLE

```
let p = mp-path("(0,0)..(10,22){up}..(24,40)..(30,20)..(18,0)..(14,14)..(30,26)..(52,8)..(66,26)")
let ink = rgb("#1d2a44")
stack(dir: ttb, spacing: 6pt,
  mp-fig(..copperplate(p, light: 0.5pt, heavy: 3pt,
    slant: 70deg, sharpness: 1.4, fill: ink), pad: 3pt),
  // no taper, heavier shade
  // sans effilage, plein plus lourd
  mp-fig(..copperplate(p, light: 0.5pt, heavy: 4.5pt,
    taper: none, fill: ink), pad: 3pt))
```



prongs

```
prongs(p, offsets, width: 0.6pt, fill: black, tol: 0.02pt)
```

Une plume à plusieurs dents (plume fendue, burin) : une ligne fine par décalage, parallèle au chemin.

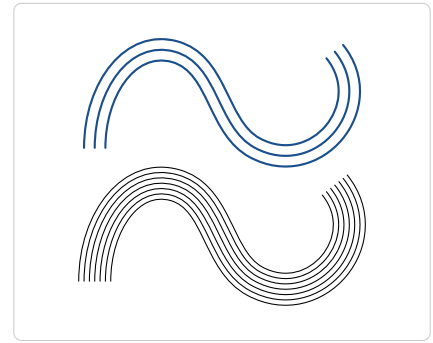
PARAMÈTRES

nom	type	défaut	description
<code>p</code>	chemin	obligatoire	Chemin.
<code>offsets</code>	tableau de longueurs	obligatoire	Décalages des dents (positif = à gauche du sens de parcours).
<code>width</code>	longueur	0.6pt	Largeur de chaque ligne.
<code>fill</code>	couleur	black	Couleur.
<code>tol</code>	longueur	0.02pt	Précision des courbes décalées.

RENVOIE un tableau d'objets dessinables.

EXEMPLE


```
let p = mp-path("(0,0){up}..(30,36)..(60,0)..(90,36)")
mp-fig(
  ..prongs(p, (-4pt, 0pt, 4pt), width: 0.8pt,
    fill: rgb("#1a4f8b")),
  ..prongs(shifted(p, 0pt, -50pt),
    range(-3, 4).map(i => i * 2pt), width: 0.4pt),
  pad: 6pt)
```



delimiter

```
delimiter(a, b, kind: "brace", amplitude: 6pt, flip: false, light: 0.4pt, heavy: 1.8pt,
  fill: black)
```

Une accolade ou une parenthèse calligraphique entre deux points, bombée à gauche de $a \rightarrow b$; la plume gonfle là où la courbe tourne. Pour les chemins nus, voir `brace-path` et `paren-path`.

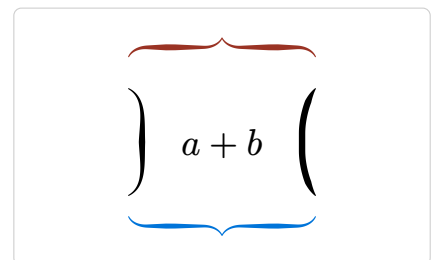
PARAMÈTRES

nom	type	défaut	description
<code>a</code>	point	obligatoire	Point de départ.
<code>b</code>	point	obligatoire	Point d'arrivée.
<code>kind</code>	chaîne	"brace"	"brace", "paren" (courbe) ou "paren-straight" (milieu droit).
<code>amplitude</code>	longueur	6pt	Importance du bombé.
<code>flip</code>	booléen	false	Bombe à droite de $a \rightarrow b$ à la place.
<code>light</code>	longueur	0.4pt	Largeur de plume aux extrémités.
<code>heavy</code>	longueur	1.8pt	Largeur de plume là où la courbe gonfle.
<code>fill</code>	couleur	black	Couleur de l'encre.

RENVOIE un tableau d'objets dessinables.

EXEMPLE

```
mp-fig(
  ..delimiter((0pt, 0pt), (70pt, 0pt), amplitude: 7pt,
    heavy: 2.4pt, fill: rgb("#9a3324")),
  ..delimiter((0pt, -12pt), (0pt, -52pt), kind: "paren",
    amplitude: 5pt, heavy: 2.4pt),
  ..delimiter((70pt, -52pt), (70pt, -12pt),
    kind: "paren-straight", amplitude: 5pt, heavy:
    2.4pt),
  // flip: the brace bulges downwards
  // flip : l'accolade bombe vers le bas
  ..delimiter((0pt, -60pt), (70pt, -60pt), flip: true,
    amplitude: 7pt, heavy: 2pt, fill: blue),
  mp-label(text(14pt)[$a+b$], (35pt, -32pt),
    pad: 4pt)
```



Version

nib-version

```
nib-version
```

La version du paquet, sous forme de chaîne. Utile dans les en-têtes et les rapports de bogue.

RENVIE une chaîne, par exemple "0.3.0".

EXEMPLE

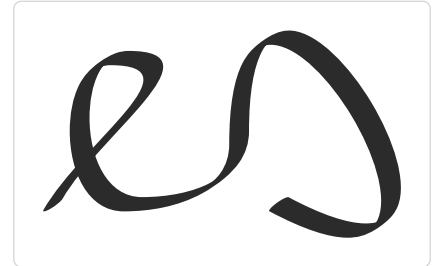
```
[nibart version: #nib-version]
```

```
nibart version: 0.3.0
```

Recettes

Boucle façon écriture à la plume plate

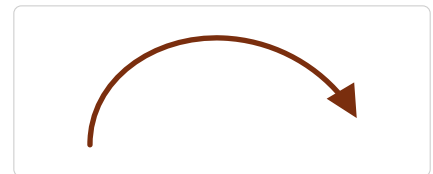
```
// one path with imposed directions, drawn with a broad nib
nib
// un chemin à directions imposées, tracé à la plume
plate
let p = mp-path("(0,0){dir 60}..(25,55){left}..(10,30)
{down}..(30,0){right}..(70,25){up}..(85,0)")
nib-stroke(p, pen: broadnib(9pt, t: 1.8pt, angle: 35deg),
  fill: rgb("#2b2b2b"), pad: 4pt)
```



Une flèche courbe

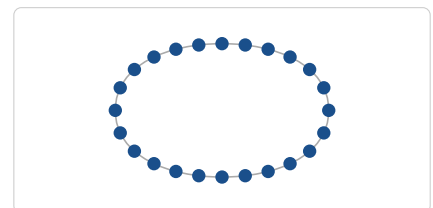
Raccourcissez la hampe pour qu'elle s'arrête à la base de la pointe, puis placez la pointe avec `frame-at` et `in-frame` :

```
let p = mp-path("(0,0){up}..(50,40)..(100,10)")
// arrowhead pointing along +x, tip at the origin
// pointe de flèche vers +x, sommet à l'origine
let head = polyline((-12pt, 6pt), (0pt, 0pt), (-12pt,
-6pt)),
  cycle: true)
let end = frame-at(p, path-length(p))
mp-fig(
  draw(shorten(p, end: 10pt), pen: pencircle(2pt),
    fill: rgb("#7a2e0e")),
  mp-fill(in-frame(head, end), fill: rgb("#7a2e0e")),
  pad: 4pt)
```



Un collier de perles sur une courbe fermée

```
let ring = mp-path("(0,0)..(40,25)..(80,0)..
(40,-25)..cycle")
// 24 beads evenly spaced by arc length
// 24 perles régulièrement espacées en abscisse
curviligne
let beads = frames-along(ring, count: 24).map(f =>
  mp-dot(f.pos, pen: pencircle(5pt), fill:
  rgb("#1a4f8b")))
mp-fig(
  draw(ring, pen: pencircle(0.6pt), fill: gray),
  ..beads, pad: 4pt)
```



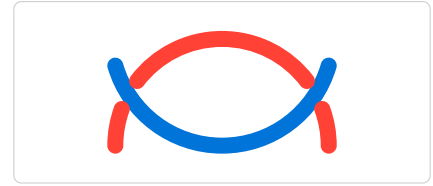
Croisements dessus/dessous à la main

`knot` le fait automatiquement ; voici le principe avec `intersection-times` et `gap-at` :

```

let a = mp-path("(0,0)..(40,40)..(80,0)")
let b = mp-path("(0,30)..(40,0)..(80,30)")
let ts = intersection-times(a, b).map(x => x.at(0))
mp-fig(
  draw(b, pen: pencircle(6pt), fill: blue),
  ..gap-at(a, ts, 6pt).map(q =>
    draw(q, pen: pencircle(6pt), fill: red)),
  pad: 4pt)

```



Reprendre un dessin écrit pour nibst

Donnez les segments cubiques à `cubics(..., y-down: true)` (nibst utilise des coordonnées y vers le bas) et traduisez les options : `width/thinness/angle/follow` → `nibpen`, `dash` → `dashes`, `pressure` → `pressure`, `layered` → `overlap: "layered"`, `debug` → `debug: true`.

Notes et limites

- L'**algorithme de Hobby** est validé contre `mpost` sur 17 spécifications (mêmes points de contrôle à 1e-2 pt près au pire, environ 6e-5 sinon) ; les opérations sur chemins concordent aussi. Le script de contrôle est dans le dépôt (`tests/cmp_hobby.py` , nécessite MetaPost).
- Les formes de trait concordent avec l'estampage brut de la plume à 0,25 pt près (testé à 576 ppi).
- Non implémentés : `atleast` , `&` (concaténation : utilisez `path-join`), `buildcycle` , et `...` (se comporte comme `..`).
- `draw` renvoie des formes **remplies**, pas des traits ; le contenu de `mp-label` n'entre pas dans la boîte englobante automatique.
- Le plugin est un fichier WebAssembly local : il fonctionne avec le CLI Typst, l'application web et les paquets de Typst Universe, mais pas dans un environnement qui interdit les plugins WebAssembly.
- **Performances.** Quelques centaines de nœuds sont instantanés ; les plumes variables, pointillés, pression et `copperplate` coûtent davantage (subdivision `refine` , une union booléenne par morceau). `knot` sur une grande surface, ou plus d'un millier de courbes dans un document, peut être lent et gourmand en mémoire : réduisez `refine` , augmentez `tol` , agrandissez `step` , ou découpez le document.
- Les erreurs commencent par `nib:` ou `nibart:` ; les plus fréquentes : `pens` avec un mauvais nombre d'entrées, une longueur attendue mais une chaîne ou des `em` donnés, une plume non elliptique avec `dash/pressure` , `shorten` qui retire plus que tout le chemin.

Crédits et licence

Licence MIT (voir `LICENSE`). Crates tierces et remerciements : `THIRD-PARTY-NOTICES.md`. Le langage de chemins et l'algorithme de Hobby viennent de Metafont/MetaPost (D. Knuth, J. D. Hobby) ; les options calligraphiques sont réimplémentées d'après le comportement décrit par le paquet `nibst` (B. Auguie) sans reprendre son code ; les idées de nœuds et d'ornements suivent les paquets LaTeX `spath3`, `knots` et `calligraphy` (A. Stacey), également sans code commun.

Auteur : FERGOUS Abdelhak <@fergousA> — **dépôt** : <https://github.com/fergousA/nibart>.

Index des fonctions

arclength	12	weld	21
arctime	12	xscaled	24
brace-path	11	yscaled	24
broadnib	28		
close-smooth	10		
closest-time	15		
copperplate	40		
crossings	39		
cubic	6		
cubics	7		
dashes	30		
delimiter	41		
direction-angle	14		
direction-of	13		
draw	32		
fit-between	22		
frame-at	17		
frames-along	18		
gap-at	21		
in-frame	17		
intersection-point	16		
intersection-times	15		
join-smooth	10		
knot	38		
mp-dot	34		
mp-fig	36		
mp-fill	34		
mp-group	36		
mp-label	35		
mp-path	5		
mp-path-pts	5		
mp-skeleton	35		
nib-stroke	33		
nib-version	41		
nibpen	29		
offset	22		
paren-path	11		
path-bbox	14		
path-close	9		
path-join	8		
path-join-all	8		
path-length	12		
pencircle	27		
penellipse	27		
penpoly	28		
penrazor	29		
pensquare	28		
point-of	13		
polyline	7		
pressure	30		
prongs	40		
reflected	25		
remove-intervals	20		
reverse	19		
rotated	25		
round-corners	9		
scaled	23		
self-intersections	16		
shifted	23		
shorten	19		
slanted	24		
split-at	20		
straight	7		
stroke-items	32		
subpath	18		
transformed	26		